# BandHiC

Weibing Wang

Jul 22, 2025

# CONTENTS

Version: 0.1.6

Welcome to the BandHiC documentation!

This documentation provides a comprehensive guide to the BandHiC library, including its features, installation instructions, and API reference. It is designed to help you efficiently store and manipulate Hi-C contact matrices in a banded form, enabling NumPy-compatible operations, matrix masking, diagonal reduction, file I/O, and domain-level interaction analysis.

If you have any questions, please contact us:

Author: Wang Weibing

Email: wangweibing@xidian.edu.cn

Useful links: Website | PyPI | Github

CHAPTER

ONE

# QUICK START

This section shows basic usage of the **BandHiC** package, including installation and a brief introduction to its features.

## 1.1 Overview

BandHiC is a Python package for efficient storage, manipulation, and analysis of Hi-C matrices using a banded matrix representation.

BandHiC adopts a banded storage scheme that stores only a configurable diagonal bandwidth of the dense Hi-C contact matrices. This design can reduce memory usage by up to 99% compared to dense matrices, while still supporting fast random access and user-friendly indexing operations.

Main features include:

- Efficient storage of Hi-C matrices using a banded representation.
- Fast random access and indexing operations.
- Support for flexible masking of missing values, outliers, and unmappable regions.
- Vectorized operations optimized with NumPy for high performance.
- Scalable for ultra-high-resolution Hi-C data analysis.

## 1.2 Installation

### 1.2.1 Required Package

**BandHiC** can be installed on Linux-like systems and requires the following dependencies:

1. python >= 3.11
2. numpy >= 2.3
3. pandas >= 2.3
4. scipy >= 1.16
5. cooler >= 0.10
6. hic_straw >= 1.3

There are two recommended ways to install **BandHiC**:

### 1.2.2 Option 1: Install via pip

If you already have Python >= 3.11 installed:

```
pip install bandhic
```

### 1.2.3 Option 2: Install from source with conda

```
# 1. Clone the repository
git clone https://github.com/xdwwb/BandHiC-Master.git
cd BandHiC-Master

# 2. Create the environment and activate it
conda env create -f environment.yml
conda activate bandhic

# 3. Install BandHiC
pip install .
```

### 1.2.4 Build Troubleshooting for hic-straw

If you encounter an error like the following while installing or building `hic-straw`:

```
fatal error: curl/curl.h: No such file or directory
```

This means the C++ extension in `hic-straw` requires the **libcurl development headers**, which are not installed by default on many systems.

**Solution 1: Install system dependencies (for pip installation)**

You need to install the `libcurl` development package before building:

- **On Ubuntu/Debian**:

  ```
  sudo apt-get update
  sudo apt-get install libcurl4-openssl-dev
  ```

- **On Fedora/CentOS/RHEL**:

  ```
  sudo dnf install libcurl-devel
  ```

- **On macOS (with Homebrew)**:

  ```
  brew install curl
  ```

  If Homebrew's curl is not found automatically, you may need to set environment variables:

  ```
  export CPATH="$(brew --prefix curl)/include"
  export LIBRARY_PATH="$(brew --prefix curl)/lib"
  ```

**Solution 2: Use Conda (recommended for convenience)**

Instead of building `hic-straw` from source, you can install a prebuilt binary via Bioconda:

```
conda install -c bioconda hic-straw
```

To avoid conflicts and ensure reproducibility, we recommend installing it in a fresh Conda environment:

```
conda create -n bandhic-env python=3.11
conda activate bandhic-env
conda install -c bioconda hic-straw

# Install BandHiC
pip install bandhic
```

## 1.3 What is BandHiC?

### 1.3.1 Data structure of BandHiC

To address the growing memory demands posed by high-resolution Hi-C data, we introduce `band_hic_matrix`, the core class implemented in the BandHiC package. For a Hi-C contact matrix $A \in \mathbb{R}^{n \times n}$ at resolution *r*, `band_hic_matrix` retains only the diagonals within a user-defined bandwidth *k*, yielding a compact representation $D \in \mathbb{R}^{n \times k}$. This format ensures that each column in *D* corresponds to a fixed diagonal of *A*, such that the mapping $A[i, j] = D[i, j - i]$ holds for $|i - j| \leq k$.

The memory efficiency achieved by this strategy is substantial. When $k \ll n$, the memory footprint of `band_hic_matrix` is reduced from $\mathcal{O}(n^2)$ to $\mathcal{O}(nk)$. For example, assuming a resolution of 1 kb and a bandwidth of 2 Mb (k = 200), the representation of chromosome 1 of the human genome (~249 Mb) requires 3.7 GB of memory, less than 1% of the memory required by the dense matrix (~461.9 GB). This compression enables the use of high-resolution Hi-C data on commodity hardware without sacrificing random access efficiency.

To further enhance flexibility of usage, `band_hic_matrix` integrates an optional two-tier masking mechanism. The element-wise mask matrix $M \in \{0, 1\}^{n \times k}$ allows users to selectively ignore missing or outlier contacts, enabling robust statistical estimation on unmasked subsets. Additionally, a bin-level mask $X \in \{0, 1\}^n$ supports the exclusion of entire rows or columns, which is particularly useful for removing repetitive genomic regions devoid of a valid Hi-C signal. These masking features facilitate downstream tasks such as the estimation of average contact intensity at given distances, while confirming statistical validity.

Lastly, a scalar default value *d* is defined to fill in the undefined entries of *A* not covered by the band matrix *D*. This default is typically set to 0, consistent with the assumption that long-range interactions are negligibly sparse. It also ensures that reconstruction of the full matrix *A* (if required) can be achieved seamlessly by combining *D*, *M*, *X*, and *d*. Overall, `band_hic_matrix` provides an efficient, flexible data structure for scalable Hi-C data analysis.

### 1.3.2 Features of BandHiC

A key feature of `band_hic_matrix` is its direct coordinate mapping between the banded matrix *B* and the full dense matrix *A*. For any pair of genomic loci *(i, j)* satisfying the band constraint $|i - j| \leq k$, the interaction frequency $A[i, j]$ can be accessed in constant time via $D[i, j - i]$. This structure ensures random access in $\mathcal{O}(1)$ time, which is critical for performance-sensitive Hi-C analyses, particularly when memory constraints prohibit the use of fully dense matrices. Data access in `band_hic_matrix` is fully consistent with that of a dense matrix, as each entry is accessed via $B[i, j] = D[i, j - i] = A[i, j]$, allowing users to interact with `band_hic_matrix` objects as if they were dense matrices without needing to consider the underlying implementation.

Owing to this random-access capability, `band_hic_matrix` supports NumPy-like indexing semantics, including slicing, boolean indexing, and fancy indexing. This design allows users to easily query local chromatin contacts. For instance, a slice operation such as `B[i:j, i:j]` retrieves a banded submatrix. Combined with the `todense` operation, this enables reconstruction of the dense submatrix for downstream analysis or visualization.

In addition to flexible data access, `band_hic_matrix` also supports a wide range of numerical operations, including element-wise arithmetic operations and reduction operations. These operations are implemented using NumPy for efficiency. Standard reduction operations such as `sum`, `min`, and `max` are supported along conventional axes (rows or columns), as well as along the diagonal axis, which is particularly useful in summarizing interaction intensities by genomic distance. This feature facilitates common Hi-C analyses such as distance-decay profiling.

Taken together, band_hic_matrix combines the memory efficiency of a banded storage model with the expressiveness of NumPy's interface. By mimicking both NumPy's ndarray and MaskedArray behaviors, it provides an intuitive and powerful interface for users, substantially lowering the barrier to adoption and promoting integration into existing Hi-C data analysis workflows.

### 1.3.3 BandHiC reduces the memory consumption of TopDom

To evaluate the effectiveness of BandHiC, we benchmarked the TopDom algorithm on chromosome 1 of mouse embryonic stem cell (mESC) Micro-C data across multiple resolutions using both dense matrix and band_hic_matrix. TopDom was reimplemented in Python to support both NumPy's ndarray and BandHiC's band_hic_matrix class. As shown in Fig. 1B, BandHiC substantially reduces memory usage at all tested resolutions. At 1000 bp and 500 bp resolution, the dense matrices require over 72 GB of memory, exceeding the available system memory (60 GB RAM and 12 GB swap space), and causing the program to terminate due to memory allocation failure. In contrast, the banded format completes successfully, with memory usage of only 5,989 MB and 23,902 MB at 1000 bp and 500 bp resolution, respectively. The banded format introduces modest additional runtime compared to the dense matrix (Fig. 1C). This overhead stems not from masking—which was disabled in this evaluation—but from index computation performed during element access in the band_hic_matrix object.

These results indicate that band_hic_matrix enables domain-calling algorithms like TopDom to be applied to high-resolution Hi-C data on standard hardware, making large-scale analysis feasible without incurring significant computational cost. Owing to BandHiC's NumPy-like API, other Hi-C-based pattern identification methods can be reimplemented with minimal modifications to the original code, thereby significantly improving their scalability and adaptability to higher-resolution data.

CHAPTER

TWO

# TUTORIALS

This section provides detailed tutorials on how to use the **BandHiC** package effectively.



## 2.1 Prerequisites

BandHiC can serve as an alternative to the NumPy package when managing and manipulating Hi-C data, aiming to address the issue of excessive memory usage caused by storing dense matrices using NumPy's `ndarray`. At the same time, BandHiC supports masking operations similar to NumPy's `ma.MaskedArray` module, with enhancements tailored for Hi-C data.

Users can leverage their experience with NumPy when using the BandHiC package, so it is recommended that users have some basic knowledge of NumPy. A link to NumPy is provided below: https://numpy.org

## 2.2 Import `bandhic` package

```
>>> import bandhic as bh
```

## 2.3 Initialize a `band_hic_matrix` object

Initialize from a SciPy `coo_matrix` object:

```
>>> import bandhic as bh
>>> import numpy as np
```

(continues on next page)

(continued from previous page)

```
>>> from scipy.sparse import coo_matrix
>>> coo = coo_matrix(([1, 2, 3], ([0, 1, 2],[0, 1, 2])), shape=(3,3))
>>> mat1 = bh.band_hic_matrix(coo, diag_num=2)
```

Initialize from a tuple (data, (row, col)):

```
>>> mat2 = bh.band_hic_matrix(([4, 5, 6], ([0, 1, 2],[2, 1, 0])), diag_num=1)
```

Initialize from a full dense array, only upper-triangular part is stored, lower part is symmetrized:

```
>>> arr = np.arange(16).reshape(4,4)
>>> mat3 = bh.band_hic_matrix(arr, diag_num=3)
```

## 2.4 Load or save a band_hic_matrix object

```
>>> bh.save_npz('./sample.npz', mat)
>>> mat = bh.load_npz('./sample.npz')
```

Load from `.hic` file:

```
>>> mat = bh.straw_chr('sample.hic',
                       'chr1',
                       resolution=10000,
                       diag_num=200
                       )
```

Load from `.mcool` file:

```
>>> mat = bh.cooler_chr('sample.mcool',
                        'chr1',
                        diag_num=200
                        resolution=10000,
                        )
```

## 2.5 Construct a band_hic_matrix object

```
# Create a band_hic_matrix object filled with zeros.
>>> mat1 = bh.zeros((5, 5), diag_num=3, dtype=float)

# Create a band_hic_matrix object filled with ones.
>>> mat2 = bh.ones((5, 5), diag_num=3, dtype=float)

# Create a band_hic_matrix object filled as an identity matrix.
>>> mat3 = bh.eye((5, 5), diag_num=3, dtype=float)

# Create a band_hic_matrix object filled with a specified value.
>>> mat4 = bh.full((5, 5), fill_value=0.1, diag_num=3, dtype=float)

# Create a band_hic_matrix object matching another matrix, filled with zeros.
>>> mat5 = bh.zeros_like(mat1, diag_num=3, dtype=float)
```

(continues on next page)

(continued from previous page)

```
# Create a band_hic_matrix object matching another matrix, filled with ones.
>>> mat6 = bh.ones_like(mat1, diag_num=3, dtype=float)

# Create a band_hic_matrix object matching another matrix, filled as an identity matrix.
>>> mat7 = bh.eye_like(mat1, diag_num=3, dtype=float)

# Create a band_hic_matrix object matching another matrix, filled with a specified value.
>>> mat8 = bh.full_like(mat1, fill_value=0.1, diag_num=3, dtype=float)
```

## 2.6 Indexing on band_hic_matrix

```
# First, we create a band_hic_matrix object:
>>> import numpy as np
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(16).reshape(4,4), diag_num=2)

# Single-element access (scalar)
>>> mat[1, 2]
6

# Masked element returns masked
>>> mat2 = bh.band_hic_matrix(np.eye(4), dtype=int, diag_num=2, mask=([0],[1]))
>>> mat2[0, 1]
masked

# Square submatrix via two-slice indexing returns band_hic_matrix
>>> sub = mat[1:3, 1:3]
>>> isinstance(sub, bh.band_hic_matrix)
True

# Single-axis slice returns band_hic_matrix for square region
>>> sub2 = mat[0:2]  # equivalent to mat[0:2, 0:2]
>>> isinstance(sub2, bh.band_hic_matrix)
True

# Fancy indexing returns ndarray or MaskedArray
>>> arr = mat[[0,2,3], [1,2,0]]
>>> isinstance(arr, np.ndarray)
True

>>> mat.add_mask([0,1],[1,2])  # Add mask to some entries
>>> masked_arr = mat[[0,1], [1,2]]
>>> isinstance(masked_arr, np.ma.MaskedArray)
True

# Boolean indexing with band_hic_matrix
>>> mat3 = bh.band_hic_matrix(np.eye(4), diag_num=2, mask=([0,1],[1,2]))
>>> bool_mask = mat3 > 0  # Create a boolean mask
>>> result = mat3[bool_mask]  # Use boolean mask for indexing
>>> isinstance(result, np.ma.MaskedArray)
```

(continues on next page)

(continued from previous page)

```
True
>>> result
masked_array(data=[1.0, 1.0, 1.0, 1.0],
             mask=[False, False, False, False],
       fill_value=0.0)
```

## 2.7 Masking

```
# Add item-wise mask:
>>> mat.add_mask([0, 1], [1, 2])

# Add row/column mask:
>>> mask = np.array([True, False, False])
>>> mat.add_mask_row_col(mask)

# Remove mask for specified indices.
>>> mat.unmask(( [0],[1] ))

# Remove all item-wise mask and row/column mask.
>>> mat.unmask()

# Remove all item-wise mask and row/column mask.
>>> mat.clear_mask()

# Drop all item-wise mask but preserve all row/column mask.
>>> mat.drop_mask()

# Drop all row/column mask.
>>> mat.drop_mask_row_col()

# Access masked `band_hic_matrix` will obtain `np.ma.MaskedArray` object:
>>> mat.add_mask([0, 1], [1, 2])
>>> masked_arr = mat[[0,1], [1,2]]
>>> isinstance(masked_arr, np.ma.MaskedArray)
True
```

## 2.8 Universal functions (ufunc)

Universal functions that BandHiC supports:

| Function 1 | Description 1 | Function 2 | Description 2 |
|---|---|---|---|
| absolute | Absolute value | add | Element-wise addition |
| arccos | Inverse cosine | arccosh | Inverse hyperbolic cosine |
| arcsin | Inverse sine | arcsinh | Inverse hyperbolic sine |
| arctan | Inverse tangent | arctan2 | Arctangent of y/x with quadrant |
| arctanh | Inverse hyperbolic tangent | bitwise_and | Element-wise bitwise AND |
| bitwise_or | Element-wise bitwise OR | bitwise_xor | Element-wise bitwise XOR |
| cbrt | Cube root | conj | Complex conjugate |

continues on next page

Table 1 – continued from previous page

| Function 1 | Description 1 | Function 2 | Description 2 |
|---|---|---|---|
| conjugate | Alias for conj | cos | Cosine function |
| cosh | Hyperbolic cosine | deg2rad | Degrees to radians |
| degrees | Radians to degrees | divide | Element-wise division |
| divmod | Quotient and remainder | equal | Element-wise equality test |
| exp | Exponential | exp2 | Base-2 exponential |
| expm1 | exp(x) - 1 | fabs | Absolute value (float) |
| float_power | Floating-point power | floor_divide | Integer division (floor) |
| fmod | Modulo operation | gcd | Greatest common divisor |
| greater | Element-wise greater-than test | greater_equal | Greater-than or equal test |
| heaviside | Heaviside step function | hypot | Euclidean norm |
| invert | Bitwise inversion | lcm | Least common multiple |
| left_shift | Bitwise left shift | less | Element-wise less-than test |
| less_equal | Less-than or equal test | log | Natural logarithm |
| log1p | log(1 + x) | log2 | Base-2 logarithm |
| log10 | Base-10 logarithm | logaddexp | log(exp(x) + exp(y)) |
| logaddexp2 | Base-2 version of logaddexp | logical_and | Element-wise logical AND |
| logical_or | Element-wise logical OR | logical_xor | Element-wise logical XOR |
| maximum | Element-wise maximum | minimum | Element-wise minimum |
| mod | Remainder (modulo) | multiply | Element-wise multiplication |
| negative | Element-wise negation | not_equal | Element-wise inequality test |
| positive | Returns input unchanged | power | Raise to power |
| rad2deg | Radians to degrees | radians | Degrees to radians |
| reciprocal | Element-wise reciprocal | remainder | Modulo remainder |
| right_shift | Bitwise right shift | rint | Round to nearest integer |
| sign | Sign of input | sin | Sine function |
| sinh | Hyperbolic sine | sqrt | Square root |
| square | Square of input | subtract | Element-wise subtraction |
| tan | Tangent function | tanh | Hyperbolic tangent |
| true_divide | Division that returns float | | |

BandHiC supports these universal functions, and they can be used in the following three ways:

1. As methods of the `band_hic_matrix` object:

```
# When two band_hic_matrix objects are involved, their shape and diag_num must match
>>> mat3 = mat1.add(mat2)
>>> mat4 = mat1.less(mat2)
>>> mat5 = mat1.negative()
```

2. Using mathematical operators:

```
>>> mat3 = mat1 + mat2
>>> mat4 = mat1 < mat2
>>> mat5 = - mat1
```

3. Calling NumPy's universal functions:

```
>>> mat3 = np.add(mat1, mat2)
>>> mat4 = np.less(mat1, mat2)
>>> mat5 = np.negative(mat1)
```

## 2.9 Other Array Functions

| Function | Description |
|---|---|
| sum | Compute the sum of all elements along the specified axis |
| prod | Compute the product of all elements along the specified axis |
| min | Return the minimum value along the specified axis |
| max | Return the maximum value along the specified axis |
| mean | Compute the arithmetic mean along the specified axis |
| var | Compute the variance (average squared deviation) |
| std | Compute the standard deviation (square root of variance) |
| ptp | Compute the range (max - min) of values along the axis |
| all | Return `True` if all elements evaluate to `True` |
| any | Return `True` if any element evaluates to `True` |
| clip | Limit values to a specified min and max range |

BandHiC supports these functions, and they can be used in the following two ways:

1. As methods of the `band_hic_matrix` object:

```
# Compute the sum of all elements including out-of-band values filled with `default_
↪value`.
>>> result0 = mat1.sum()

# Compute the sum of all elements along the `row` axis
>>> result1 = mat1.sum(axis=0)
>>> result1 = mat1.sum(axis='row')

# Compute the sum of all elements along the `diag` axis
>>> result2 = mat1.sum(axis='diag')
```

2. Calling NumPy's functions:

```
# Compute the sum of all elements including out-of-band values filled with `default_
↪value`.
>>> result0 = np.sum(mat1)

# Compute the sum of all elements along the `row` axis
>>> result1 = np.sum(mat1, axis=0)

# Compute the sum of all elements along the `diag` axis
>>> result2 = np.sum(mat1, axis='diag')
```

CHAPTER

# THREE

# API REFERENCE

This page provides a full API overview of the BandHiC module, including the core band_hic_matrix class and its utilities.

The *bandhic* module defines data structures and utilities for efficiently storing and manipulating Hi-C contact matrices in a banded form. It enables NumPy-compatible operations, matrix masking, diagonal reduction, file I/O, and domain-level interaction analysis.

Core components:

- *band_hic_matrix*: A memory-efficient matrix class supporting banded Hi-C data.
- Utility functions: loaders (*cooler_chr*) and constructors (*ones*, *zeros*, etc.).

## 3.1 band_hic_matrix Class

The band_hic_matrix class is the core data structure in the BandHiC package, designed for efficient storage and manipulation of Hi-C contact matrices using a banded representation. It supports various operations similar to those in NumPy, while optimizing memory usage and performance.



### 3.1.1 Class Overview and Initialization

**class** bandhic.**band_hic_matrix**(*contacts*, *diag_num=1*, *mask_row_col=None*, *mask=None*, *dtype=None*, *default_value=0*, *band_data_input=False*)

Symmetric banded matrix stored in upper-triangular format. This storage format is motivated by high-resolution Hi-C data characteristics:

1. Symmetry of contact maps.
2. Interaction frequency concentrated near the diagonal; long-range contacts are sparse (mostly zero).

3. Contact frequency decays sharply with genomic distance.

By storing only the main and a fixed number of super-diagonals as columns of a band matrix (diagonal-major storage: diagonal k stored in column k), we drastically reduce memory usage while enabling random access to Hi-C contacts. Additionally, mask and mask_row_col arrays track invalid or masked contacts to support downstream analysis.

Operations on this band_hic_matrix are as simple as on a numpy.ndarray; users can ignore these storage details.

This class stores only the main diagonal and up to (diag_num - 1) super-diagonals, exploiting symmetry by mirroring values for lower-triangular access.

**Parameters**

- **contacts** (*coo_array | coo_matrix | tuple | ndarray*)
- **diag_num** (*int*)
- **mask_row_col** (*ndarray | None*)
- **mask** (*Tuple[ndarray, ndarray] | None*)
- **dtype** (*type | None*)
- **default_value** (*int | float*)
- **band_data_input** (*bool*)

**shape**

Shape of the original full Hi-C contact matrix (bin_num, bin_num), regardless of internal band storage format.

**Type**
tuple of int

**dtype**

Data type of the matrix elements, compatible with numpy dtypes.

**Type**
data-type

**diag_num**

Number of diagonals stored.

**Type**
int

**bin_num**

Number of bins (rows/columns) of the Hi-C matrix.

**Type**
int

**data**

Array of shape (*bin_num*, *diag_num*) storing banded Hi-C data.

**Type**
ndarray

**mask**

Mask for individual invalid entries. Stored as a boolean ndarray of shape (*bin_num*, *diag_num*) with the same shape as *data*.

**Type**
ndarray of bool or None

**mask_row_col**

Mask for entire rows and corresponding columns, indicating invalid bins. Stored as a boolean ndarray of shape (*bin_num*,). For computational convenience, row/column masks are also applied to the *mask* array to track masked entries.

**Type**
ndarray of bool or None

**default_value**

Default value for out-of-band entries. Entries out of the banded region and not stored in the data array will be set to this value.

**Type**
scalar

## Examples

```
>>> import bandhic as bh
>>> import numpy as np
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.shape
(4, 4)
```

**__init__**(*contacts*, *diag_num=1*, *mask_row_col=None*, *mask=None*, *dtype=None*, *default_value=0*, *band_data_input=False*)

Initialize a band_hic_matrix instance.

**Parameters**

- **contacts** (*{coo_array, coo_matrix, tuple, ndarray}*) – Input Hi-C data in COO format, tuple (data, (row, col)), full square array, or banded stored ndarray. For non-symmetric full arrays, only the upper-triangular part is used and the matrix is symmetrized. Full square arrays are not recommended for large matrices due to memory constraints.
- **diag_num** (*int, optional*) – Number of diagonals to store. Must be >=1 and <= matrix dimension. Default is 1.
- **mask_row_col** (*ndarray of bool or indices, optional*) – Mask for invalid rows/columns. Can be specified as:
  - A boolean array of shape (bin_num,) indicating which rows/columns to mask.
  - A list of indices to mask.

  Defaults to None (no masking).
- **mask** (*ndarray pair of (row_indices, col_indices), optional*) – Mask for invalid matrix entries. Can be specified as:
  - A tuple of two ndarray (row_indices, col_indices) listing positions to mask.

  Defaults to None (no masking).
- **dtype** (*data-type, optional*) – Desired numpy dtype; defaults to 'contacts' data dtype; compatible with numpy dtypes.
- **default_value** (*scalar, optional*) – Default value for unstored out-of-band entries. Default is 0.

- **band_data_input** (*bool, optional*) – If True, contacts is treated as precomputed band storage. Default is False.

**Raises**

**ValueError** – If contacts type is invalid, diag_num out of range, or array shape invalid.

**Return type**

None

### Examples

Initialize from a SciPy COO matrix:

```
>>> import bandhic as bh
>>> import numpy as np
>>> from scipy.sparse import coo_matrix
>>> coo = coo_matrix(([1, 2, 3], ([0, 1, 2],[0, 1, 2])), shape=(3,3))
>>> mat1 = bh.band_hic_matrix(coo, diag_num=2)
>>> mat1.data.shape
(3, 2)
```

Initialize from a tuple (data, (row, col)):

```
>>> mat2 = bh.band_hic_matrix(([4, 5, 6], ([0, 1, 2],[2, 1, 0])), diag_num=1)
>>> mat2.data.shape
(3, 1)
```

Initialize from a full dense array, only upper-triangular part is stored, lower part is symmetrized:

```
>>> arr = np.arange(16).reshape(4,4)
>>> mat3 = bh.band_hic_matrix(arr, diag_num=3)
>>> mat3.data.shape
(4, 3)
```

Initialize with row/column mask, this masks entire rows and corresponding columns:

```
>>> mask = np.array([True, False, False, True])
>>> mat4 = bh.band_hic_matrix(arr, diag_num=2, mask_row_col=mask)
>>> mat4.mask_row_col
array([ True, False, False,  True])
```

*mask_row_col* is also supported as a list of indices:

```
>>> mat4 = bh.band_hic_matrix(arr, diag_num=2, mask_row_col=[0, 3])
>>> mat4.mask_row_col
array([ True, False, False,  True])
```

Initialize from precomputed banded storage:

```
>>> band = mat3.data.copy()
>>> mat5 = bh.band_hic_matrix(band, band_data_input=True)
>>> mat5.data.shape
(4, 3)
```

### 3.1.2 Masking Methods

These methods allow for the application and manipulation of masks within the matrix. Masks can be used to exclude or highlight specific parts of the matrix during computations.

| | |
|---|---|
| *band_hic_matrix.init_mask*() | Initialize mask for invalid entries based on matrix shape. |
| *band_hic_matrix.add_mask*(row_idx, col_idx) | Add mask entries for specified indices. |
| *band_hic_matrix.get_mask*() | Get current mask array. |
| *band_hic_matrix.unmask*([indices]) | Remove mask entries for specified indices or clear all. |
| *band_hic_matrix.drop_mask*() | Clear the current mask by entry-level, but retain the row/column mask. |
| *band_hic_matrix.add_mask_row_col*(mask_row_col | Mask entire rows and corresponding columns. |
| *band_hic_matrix.get_mask_row_col*() | Get current row/column mask. |
| *band_hic_matrix.drop_mask_row_col*() | Clear the current row/column mask. |
| *band_hic_matrix.clear_mask*() | Clear all masks (both entry-level and row/column-level). |
| *band_hic_matrix.count_masked*() | Count the number of masked entries in the banded matrix. |
| *band_hic_matrix.count_unmasked*() | Count the number of unmasked entries in the banded matrix. |
| *band_hic_matrix.count_in_band_masked*() | Count the number of masked entries in the in-band region. |
| *band_hic_matrix.count_out_band_masked*() | Count the number of masked entries in the out-of-band region. |
| *band_hic_matrix.count_in_band*() | Count the number of valid entries in the in-band region. |
| *band_hic_matrix.count_out_band*() | Count the number of valid entries in the out-of-band region. |

#### 3.1.2.1 bandhic.band_hic_matrix.init_mask

band_hic_matrix.**init_mask**()

Initialize mask for invalid entries based on matrix shape.

**Raises**

**ValueError** – If mask is already initialized.

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.init_mask()
```

#### 3.1.2.2 bandhic.band_hic_matrix.add_mask

band_hic_matrix.**add_mask**(*row_idx*, *col_idx*)

Add mask entries for specified indices.

**Parameters**

- **row_idx** (*array-like of int*) – Row indices to mask.
- **col_idx** (*array-like of int*) – Column indices to mask.

**Raises**

**ValueError** – If row_idx and col_idx have different shapes.

**Return type**
None

### Examples

```
>>> import bandhic as bh
>>> import numpy as np
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.add_mask([0, 1], [1, 2])
```

#### 3.1.2.3 bandhic.band_hic_matrix.get_mask

band_hic_matrix.**get_mask**()

Get current mask array.

**Returns**
Current mask array or None if no mask.

**Return type**
ndarray or None

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mask = mat.get_mask()
```

#### 3.1.2.4 bandhic.band_hic_matrix.unmask

band_hic_matrix.**unmask**(*indices=None*)

Remove mask entries for specified indices or clear all.

**Parameters**
**indices** (*tuple of array-like or None*) – Tuple (row_idx, col_idx) to unmask, or None to clear all.

**Raises**
**Warning** – If no mask exists when trying to remove.

**Return type**
None

### Notes

If *indices* is None, this will clear all masks (both entry-level and row/column), equivalent to *clear_mask()*.

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(3), diag_num=2)
>>> mat.unmask(( [0],[0] ))
>>> mat.unmask()
```

#### 3.1.2.5 bandhic.band_hic_matrix.drop_mask

band_hic_matrix.**drop_mask**()

Clear the current mask by entry-level, but retain the row/column mask.

**Return type**
None

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2, mask=(np.array([[0, 1], [1,␣
↪2]])))
>>> mat.drop_mask()
```

### Notes

This method is useful when you want to keep the row/column mask but clear the entry-level mask. It will not affect the row/column mask, allowing you to maintain the masking of entire rows and columns.

#### 3.1.2.6 bandhic.band_hic_matrix.add_mask_row_col

band_hic_matrix.**add_mask_row_col**(*mask_row_col*)

Mask entire rows and corresponding columns.

**Parameters**
**mask_row_col** (*array-like of int or bool*) – If boolean array of shape (bin_num,), True entries indicate rows/columns to mask. If integer array or sequence, treated as indices of rows/columns to mask.

**Raises**
**ValueError** – If integer indices are out of bounds.

**Return type**
None

### Examples

```
>>> import bandhic as bh
>>> mask = np.array([True, False, False])
>>> mat = bh.band_hic_matrix(np.eye(3), diag_num=2)
>>> mat.add_mask_row_col(mask)
```

#### 3.1.2.7 bandhic.band_hic_matrix.get_mask_row_col

band_hic_matrix.**get_mask_row_col**()

Get current row/column mask.

**Returns**
Current row/column mask or None if no mask.

**Return type**
ndarray or None

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mask_row_col = mat.get_mask_row_col()
```

#### 3.1.2.8 bandhic.band_hic_matrix.drop_mask_row_col

band_hic_matrix.**drop_mask_row_col**()

Clear the current row/column mask.

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.drop_mask_row_col()
```

#### 3.1.2.9 bandhic.band_hic_matrix.clear_mask

band_hic_matrix.**clear_mask**()

Clear all masks (both entry-level and row/column-level).

**Return type**
None

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.clear_mask()
```

#### 3.1.2.10 bandhic.band_hic_matrix.count_masked

band_hic_matrix.**count_masked**()

Count the number of masked entries in the banded matrix.

This counts the number of entries in the upper triangular part of the matrix, excluding the diagonal and valid entries.

**Returns**
Number of valid entries in the banded matrix.

**Return type**
int

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.count_masked()
0
```

#### 3.1.2.11 bandhic.band_hic_matrix.count_unmasked

band_hic_matrix.**count_unmasked**()

Count the number of unmasked entries in the banded matrix.

**Returns**
Number of unmasked entries in the banded matrix.

**Return type**
int

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.count_unmasked()
16
```

#### 3.1.2.12 bandhic.band_hic_matrix.count_in_band_masked

band_hic_matrix.**count_in_band_masked**()

Count the number of masked entries in the in-band region.

**Returns**
Number of masked entries in the in-band region.

**Return type**
int

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.count_in_band_masked()
0
```

#### 3.1.2.13 bandhic.band_hic_matrix.count_out_band_masked

band_hic_matrix.**count_out_band_masked**()

Count the number of masked entries in the out-of-band region.

**Returns**
Number of masked entries in the out-of-band region.

**Return type**
int

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.count_out_band_masked()
0
```

#### 3.1.2.14 bandhic.band_hic_matrix.count_in_band

band_hic_matrix.**count_in_band**()

Count the number of valid entries in the in-band region.

**Returns**
Number of valid entries in the in-band region.

**Return type**
int

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.count_in_band()
10
```

#### 3.1.2.15 bandhic.band_hic_matrix.count_out_band

band_hic_matrix.**count_out_band**()

Count the number of valid entries in the out-of-band region.

**Returns**
Number of valid entries in the out-of-band region.

**Return type**
int

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2)
>>> mat.count_out_band()
6
```

### 3.1.3 Data Indexing and Modification

The following methods provide functionality to access, modify, and index the matrix data. These are essential for manipulating individual elements or subsets of the matrix.

| | |
|---|---|
| *band_hic_matrix.__getitem__*(index) | Retrieve matrix entries or submatrix using NumPy-like indexing. |
| *band_hic_matrix.__setitem__*(index, values) | Assign values to matrix entries using NumPy-like indexing. |
| *band_hic_matrix.get_values*(row_idx, col_idx) | Retrieve values considering mask. |
| *band_hic_matrix.set_values*(row_idx, col_idx, ...) | Set values at specified row and column indices. |
| *band_hic_matrix.diag*(k) | Retrieve the k-th diagonal from the matrix. |
| *band_hic_matrix.set_diag*(k, values) | Set values in the k-th diagonal of the matrix. |
| *band_hic_matrix.extract_row*(idx[, ...]) | Extract stored, unmasked band values for a given row or column. |
| *band_hic_matrix.__iter__*() | Iterate over diagonals of the matrix. |
| *band_hic_matrix.iterwindows*(width[, step]) | Iterate over the diagonals of the matrix with a specified window size. |

continues on next page

Table 2 – continued from previous page

| | |
|---|---|
| band_hic_matrix.iterrows() | Iterate over the rows of the band_hic_matrix object. |
| band_hic_matrix.itercols() | Iterate over the columns of the band_hic_matrix object. |
| band_hic_matrix.filled([fill_value, copy]) | Fill masked entries in data with default value. |
| band_hic_matrix.clip(min_val, max_val) | Clip data values to given range. |

#### 3.1.3.1 bandhic.band_hic_matrix.__getitem__

band_hic_matrix.**__getitem__**(*index*)

Retrieve matrix entries or submatrix using NumPy-like indexing.

Supports:

- Integer indexing: mat[i, j] returns a single value.
- Slice indexing: mat[i:j, i:j] returns a band_hic_matrix for square slices.
- Single-axis slice: mat[i:j] returns a band_hic_matrix same as mat[i:j, i:j].
- Fancy (array) indexing: mat[[i1, i2], [j1, j2]] returns an ndarray or MaskedArray according to *mask*.
- Mixed indexing: combinations of integer, slice, and array-like indices.
- Boolean indexing: 'band_hic_matrix' object with dtype *bool* can be used to index entries.

When both row and column indices specify the same slice (or a single slice is provided), a new band_hic_matrix representing that square submatrix is returned. New submatrix is the view of the original matrix, sharing the same data and mask. If the mask or data is altered in the submatrix, the original matrix will reflect those changes as well.

- If a single integer index is provided for both row and column, a scalar value is returned.
- If a mask is set, masked entries will return as *numpy.ma.masked* for scalars, or as a *numpy.ma.MaskedArray* for arrays.
- If a mask is not set, the scalar value is returned directly, or a numpy.ndarray for arrays.
- If a square slice is provided, a new band_hic_matrix is returned with the same diagonal number and shape as the original matrix.
- If a single slice is provided, it returns a band_hic_matrix with the same diagonal number and shape as the original matrix.
- If fancy indexing is used, it returns a numpy.ndarray or numpy.ma.MaskedArray depending on whether the mask is set.
- In all other cases, a numpy.ndarray (if no mask) or numpy.ma.MaskedArray (if mask present) is returned.

**Parameters**

**index** (*int, slice,* band_hic_matrix*, array-like of int, or tuple of these*) – index expression for rows and columns. May be:

- A pair *(row_idx, col_idx)* of ints, slices, or array-like for mixed indexing.
- A single slice selecting a square region.
- A *band_hic_matrix* object with dtype of *bool* for boolean indexing.

**Returns**

- scalar : when both row and column are integer indices.
- numpy.ndarray : for fancy or mixed indexing without mask.

- numpy.ma.MaskedArray : for fancy or mixed indexing when mask is set.
- band_hic_matrix : for square slice results.

**Return type**
scalar or ndarray or MaskedArray or *band_hic_matrix*

**Raises**
**ValueError** – If a slice step is not 1, or if indices are out of bounds.

### Examples

```
>>> import numpy as np
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(16).reshape(4,4), diag_num=2)
```

# Single-element access (scalar)

```
>>> mat[1, 2]
6
```

# Masked element returns masked

```
>>> mat2 = bh.band_hic_matrix(np.eye(4), dtype=int, diag_num=2, mask=([0],[1]))
>>> mat2[0, 1]
masked
```

# Square submatrix via two-slice indexing returns band_hic_matrix

```
>>> sub = mat[1:3, 1:3]
>>> isinstance(sub, bh.band_hic_matrix)
True
```

# Single-axis slice returns band_hic_matrix for square region

```
>>> sub2 = mat[0:2]  # equivalent to mat[0:2, 0:2]
>>> isinstance(sub2, bh.band_hic_matrix)
True
```

# Fancy indexing returns ndarray or MaskedArray

```
>>> arr = mat[[0,2,3], [1,2,0]]
>>> isinstance(arr, np.ndarray)
True
```

```
>>> mat.add_mask([0,1],[1,2])  # Add mask to some entries
>>> masked_arr = mat[[0,1], [1,2]]
>>> isinstance(masked_arr, np.ma.MaskedArray)
True
```

# Boolean indexing with band_hic_matrix

```
>>> mat3 = bh.band_hic_matrix(np.eye(4), diag_num=2, mask=([0,1],[1,2]))
>>> bool_mask = mat3 > 0  # Create a boolean mask
>>> result = mat3[bool_mask]  # Use boolean mask for indexing
```

(continues on next page)

(continued from previous page)

```
>>> isinstance(result, np.ma.MaskedArray)
True
```

```
>>> result
masked_array(data=[1.0, 1.0, 1.0, 1.0],
             mask=[False, False, False, False],
    fill_value=0.0)
```

#### 3.1.3.2 bandhic.band_hic_matrix.__setitem__

band_hic_matrix.**__setitem__**(*index*, *values*)

Assign values to matrix entries using NumPy-like indexing.

**Parameters**

- **index** (*int, tuple of (row_idx, col_idx), slice, or* band_hic_matrix) – Index expression for rows and columns. May be:
- **column.** (*- A single integer for both row and*)
- **int** (*- A tuple of row and column indices (can be*)
- **slice**
- **array-like).** (*or*)
- **region.** (*- A single slice selecting a square*)
- **indexing.** (*- A band_hic_matrix object with dtype of bool for boolean*)
- **values** (*scalar or array-like*) – Values to assign. Can be a single scalar or an array-like object.

**Raises**

- **ValueError** – If index is a slice with step not equal to 1, or if indices exceed matrix dimensions.
- **TypeError** – If *values* is not a scalar or array-like object.
- **Supports:** –
- **- Integer indexing** – mat[i, j] = value assigns to a single element.:
- **- Slice indexing** – mat[i:j, i:j] = array or scalar assigns to a square submatrix.:
- **- Single-axis slice** – mat[i:j] = ... is equivalent to mat[i:j, i:j].:
- **- Fancy (array) indexing** – mat[[i1, i2], [j1, j2]] = array or scalar for scattered assignments.:
- **- Mixed indexing** – combinations of integer, slice, and array-like indices.:
- **- Boolean indexing** – boolean mask (another band_hic_matrix with dtype=bool) selects entries to set.:

**Return type**

None

### Examples

```
>>> import bandhic as bh
>>> import numpy as np
>>> mat = bh.band_hic_matrix(np.zeros((4,4)), diag_num=2, dtype=int)
```

# Single element assignment

```
>>> mat[1, 2] = 5
>>> mat[1, 2]
5
```

# Slice assignment to square submatrix

```
>>> mat[0:2, 0:2] = [[1, 2], [2, 4]]
>>> mat[0:2, 0:2].todense()
array([[1, 2],
       [2, 4]])
```

# Single-axis slice assignment (equivalent square slice)

```
>>> mat[2:4] = 0
>>> mat[2:4].todense()
array([[0, 0],
       [0, 0]])
```

# Fancy indexing for scattered assignments

```
>>> mat[[0, 3], [1, 2]] = [7, 8]
>>> mat[0, 1], mat[3, 2]
(7, 8)
```

# Boolean mask assignment

```
>>> mat2 = bh.band_hic_matrix(np.eye(4), diag_num=2, dtype=int)
>>> bool_mask = mat2 > 0
>>> mat2[bool_mask] = 9
>>> mat2.todense()
array([[9, 0, 0, 0],
      [0, 9, 0, 0],
      [0, 0, 9, 0],
      [0, 0, 0, 9]])
```

### Notes

- Assigning to masked entries updates underlying data but does not automatically unmask.
- For multidimensional assignments, scalar values broadcast to all selected positions, while array values must match the number of targeted elements.
- If a boolean mask is used, it must be a *band_hic_matrix* with dtype *bool* and the same shape as the original matrix.
- If a single slice is provided, it behaves like mat[i:j, i:j] for square submatrices.

#### 3.1.3.3 bandhic.band_hic_matrix.get_values

band_hic_matrix.**get_values**(*row_idx*, *col_idx*)

Retrieve values considering mask.

**Parameters**

- **row_idx** (*array-like of int*) – Row indices.
- **col_idx** (*array-like of int*) – Column indices.

**Returns**

Retrieved values; masked entries yield masked results.

**Return type**

ndarray or MaskedArray

**Raises**

**ValueError** – If indices exceed matrix dimensions.

### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> mat.get_values([0,1],[1,2])
array([1., 1.])
```

#### 3.1.3.4 bandhic.band_hic_matrix.set_values

band_hic_matrix.**set_values**(*row_idx*, *col_idx*, *values*)

Set values at specified row and column indices.

**Parameters**

- **row_idx** (*array-like of int*) – Row indices where values will be set.
- **col_idx** (*array-like of int*) – Column indices where values will be set.
- **values** (*scalar or array-like*) – Values to assign.

**Return type**

None

### Examples

```
>>> import bandhic as bh
>>> mat = bh.zeros((3,3), diag_num=2, dtype=int)
>>> mat.set_values([0,1], [1,2], [4,5])
>>> mat[0,1]
4
```

### Notes

Writing to masked positions will update the underlying data but will not clear the mask.

#### 3.1.3.5 bandhic.band_hic_matrix.diag

band_hic_matrix.**diag**(*k*)

Retrieve the k-th diagonal from the matrix.

**Parameters**
**k** (*int*) – The diagonal index to retrieve.

**Returns**
The k-th diagonal values; masked if mask is set.

**Return type**
ndarray or MaskedArray

##### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> mat.diag(1)
array([1., 1.])
```

#### 3.1.3.6 bandhic.band_hic_matrix.set_diag

band_hic_matrix.**set_diag**(*k*, *values*)

Set values in the k-th diagonal of the matrix.

**Parameters**

- **k** (*int*) – The diagonal index to set.
- **values** (*array-like*) – The values to set in the diagonal.

**Raises**
**ValueError** – If k is out of range or values length mismatch.

**Return type**
None

##### Examples

```
>>> import bandhic as bh
>>> mat = bh.zeros((4,4), diag_num=3)
>>> mat.set_diag(1, [9,9,9])
>>> mat.diag(1)
array([9., 9., 9.])
```

#### 3.1.3.7 bandhic.band_hic_matrix.extract_row

band_hic_matrix.**extract_row**(*idx*, *extract_out_of_band=True*)

Extract stored, unmasked band values for a given row or column.

**Parameters**

- **idx** (*int*) – Row (or column, due to symmetry) index for which to extract band values.
- **extract_out_of_band** (*bool, optional*) – If True, include out-of-band entries filled with *default_value* and masked if appropriate. If False (default), return only the stored band values.

**Returns**
If *extract_out_of_band=False*, a 1D array of length up to *diag_num* containing band values. If *extract_out_of_band=True*, a 1D array of length *bin_num* with all row/column values.

**Return type**
ndarray or MaskedArray

**Raises**
**ValueError** – If *idx* is outside the range [0, bin_num-1].

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(9).reshape(3,3), diag_num=2)
>>> mat.extract_row(0, extract_out_of_band=False)
array([0, 1])
>>> mat.extract_row(0)
array([0, 1, 0])
```

#### 3.1.3.8 bandhic.band_hic_matrix.__iter__

band_hic_matrix.**__iter__**()

Iterate over diagonals of the matrix.

**Yields**
*ndarray* – Values of each diagonal.

**Return type**
Iterator[ndarray]

### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> for band in mat:
...     print(band)
[1. 1. 1.]
[1. 1.]
```

#### 3.1.3.9 bandhic.band_hic_matrix.iterwindows

band_hic_matrix.**iterwindows**(*width*, *step=1*)

Iterate over the diagonals of the matrix with a specified window size.

**Parameters**

- **width** (*int*) – The size of the window to iterate over.
- **step** (*int, optional*) – Step size between windows. Default is 1.

**Yields**
*band_hic_matrix* – The values in the current window.

**Return type**
Iterator[*band_hic_matrix*]

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> for win in mat.iterwindows(2):
...     print(win)
band_hic_matrix(shape=(2, 2), diag_num=2, dtype=<class 'numpy.float64'>)
band_hic_matrix(shape=(2, 2), diag_num=2, dtype=<class 'numpy.float64'>)
```

### 3.1.3.10 bandhic.band_hic_matrix.iterrows

band_hic_matrix.**iterrows**()

Iterate over the rows of the band_hic_matrix object.

**Yields**
*ndarray* – The values in the current row.

**Return type**
Iterator[ndarray]

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> for row in mat.iterrows():
...     print(row)
[1. 1. 1.]
[1. 1. 1.]
[1. 1. 1.]
```

### 3.1.3.11 bandhic.band_hic_matrix.itercols

band_hic_matrix.**itercols**()

Iterate over the columns of the band_hic_matrix object.

**Yields**
*ndarray* – The values in the current column.

**Return type**
Iterator[ndarray]

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> for col in mat.itercols():
...     print(col)
[1. 1. 1.]
[1. 1. 1.]
[1. 1. 1.]
```

#### 3.1.3.12 bandhic.band_hic_matrix.filled

band_hic_matrix.**filled**(*fill_value=None*, *copy=True*)

Fill masked entries in data with default value.

**Parameters**

- **fill_value** (*scalar*) – Value to assign to masked entries.
- **copy** (*bool*)

**Raises**

**ValueError** – If no mask is initialized.

**Return type**

*band_hic_matrix*

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(4), diag_num=2, mask = ([0,1],[1,2]))
>>> mat.filled()
    band_hic_matrix(shape=(4, 4), diag_num=2, dtype=float64)
```

#### 3.1.3.13 bandhic.band_hic_matrix.clip

band_hic_matrix.**clip**(*min_val*, *max_val*)

Clip data values to given range.

**Parameters**

- **min_val** (*scalar*)
- **max_val** (*scalar*)

**Return type**

None

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> mat = mat.clip(0, 10)
```

### 3.1.4 Data Reduction and Normalization

These methods enable various data reduction operations, such as summing, averaging, or aggregating matrix values along specific axes or dimensions.

| | |
|---|---|
| *band_hic_matrix.min*([axis]) | Compute the minimum value in the matrix or along a given axis. |
| *band_hic_matrix.max*([axis]) | Compute the maximum value in the matrix or along a given axis. |
| *band_hic_matrix.sum*([axis]) | Compute the sum of the values in the matrix or along a given axis. |

continues on next page

Table 3 – continued from previous page

| | |
|---|---|
| band_hic_matrix.mean([axis]) | Compute the mean value of the matrix or along a given axis. |
| band_hic_matrix.std([axis]) | Compute the standard deviation of the values in the matrix or along a given axis. |
| band_hic_matrix.var([axis]) | Compute the variance of the values in the matrix or along a given axis. |
| band_hic_matrix.prod([axis]) | Compute the product of the values in the matrix or along a given axis. |
| band_hic_matrix.ptp([axis]) | Compute the peak-to-peak (maximum - minimum) value of the matrix or along a given axis. |
| band_hic_matrix.all([axis, banded_only]) | Test whether all (or any) array elements along a given axis evaluate to True. |
| band_hic_matrix.any([axis, banded_only]) | Test whether all (or any) array elements along a given axis evaluate to True. |
| band_hic_matrix.normalize([inplace]) | Normalize each diagonal of the matrix to have zero mean and unit variance. |

#### 3.1.4.1 bandhic.band_hic_matrix.min

band_hic_matrix.**min**(*axis=None*)

Compute the minimum value in the matrix or along a given axis.

**Parameters**

**axis** (*None, int, or {'row','col','diag'}, optional*) – Axis along which to compute the minimum:

- None: compute over all stored values (and default for missing).
- 0 or 'row': per-row reduction.
- 1 or 'col': per-column reduction.
- 'diag': per-diagonal reduction.

Default is None.

**Returns**

Minimum value(s) along the specified axis.

**Return type**

scalar or ndarray

**Raises**

**ValueError** – If axis is not one of the supported values.

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(9).reshape(3,3), diag_num=2)
>>> mat.min()  # global minimum
0
>>> mat.min(axis='row')
array([0, 1, 0])
```

#### 3.1.4.2 bandhic.band_hic_matrix.max

band_hic_matrix.**max**(*axis=None*)

Compute the maximum value in the matrix or along a given axis.

**Parameters**

**axis** (*None, int, or {'row','col','diag'}, optional*) – Axis along which to compute the maximum:

- None: compute over all stored values (and default for missing).
- 0 or 'row': per-row reduction.
- 1 or 'col': per-column reduction.
- 'diag': per-diagonal reduction.

Default is None.

**Returns**

Maximum value(s) along the specified axis.

**Return type**

scalar or ndarray

**Raises**

**ValueError** – If axis is not one of the supported values.

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(9).reshape(3,3), diag_num=2)
>>> mat.max()  # global maximum
8
>>> mat.max(axis='row')
array([1, 5, 8])
```

#### 3.1.4.3 bandhic.band_hic_matrix.sum

band_hic_matrix.**sum**(*axis=None*)

Compute the sum of the values in the matrix or along a given axis.

**Parameters**

**axis** (*None, int, or {'row','col','diag'}, optional*) – Axis along which to compute the sum:

- None: sum over all stored values (and default for missing).
- 0 or 'row': per-row reduction.
- 1 or 'col': per-column reduction.
- 'diag': per-diagonal reduction.

Default is None.

**Returns**

Sum(s) along the specified axis.

**Return type**

scalar or ndarray

**Raises**

**ValueError** – If axis is not one of the supported values.

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(9).reshape(3,3), diag_num=2)
>>> mat.sum()  # sum of all elements
24
>>> mat.sum(axis='row')
array([ 1, 10, 13])
```

#### 3.1.4.4 bandhic.band_hic_matrix.mean

band_hic_matrix.**mean**(*axis=None*)

Compute the mean value of the matrix or along a given axis.

**Parameters**

**axis** (*None, int, or {'row','col','diag'}, optional*) – Axis along which to compute the mean:

- None: mean over all stored values (and default for missing).
- 0 or 'row': per-row reduction.
- 1 or 'col': per-column reduction.
- 'diag': per-diagonal reduction.

Default is None.

**Returns**

Mean value(s) along the specified axis.

**Return type**

scalar or ndarray

**Raises**

**ValueError** – If axis is not one of the supported values.

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(9).reshape(3,3), diag_num=2)
>>> mat.mean()  # mean of all elements
2.6666666666666665
>>> mat.mean(axis='diag')
array([4., 3.])
```

#### 3.1.4.5 bandhic.band_hic_matrix.std

band_hic_matrix.**std**(*axis=None*)

Compute the standard deviation of the values in the matrix or along a given axis.

**Parameters**

**axis** (*None, int, or {'row','col','diag'}, optional*) – Axis along which to compute the standard deviation:

- None: std over all stored values (and default for missing).

- 0 or ‘row’: per-row reduction.
- 1 or ‘col’: per-column reduction.
- ’diag’: per-diagonal reduction.

Default is None.

**Returns**
Standard deviation(s) along the specified axis.

**Return type**
scalar or ndarray

**Raises**
**ValueError** – If axis is not one of the supported values.

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(9).reshape(3,3), diag_num=2)
>>> mat.std()  # std of all elements
2.748737083745107
>>> mat.std(axis='diag')
array([3.26598632, 2.        ])
```

#### 3.1.4.6 bandhic.band_hic_matrix.var

band_hic_matrix.**var**(*axis=None*)

Compute the variance of the values in the matrix or along a given axis.

**Parameters**
**axis** (*None, int, or {'row','col','diag'}, optional*) – Axis along which to compute the variance:

- None: variance over all stored values (and default for missing).
- 0 or ‘row’: per-row reduction.
- 1 or ‘col’: per-column reduction.
- ’diag’: per-diagonal reduction.

Default is None.

**Returns**
Variance(s) along the specified axis.

**Return type**
scalar or ndarray

**Raises**
**ValueError** – If axis is not one of the supported values.

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(9).reshape(3,3), diag_num=2)
>>> mat.var()  # variance of all elements
7.555555555555555
```

(continues on next page)

(continued from previous page)

```
>>> mat.var(axis='row')
array([ 0.22222222,  2.88888889, 10.88888889])
```

#### 3.1.4.7 bandhic.band_hic_matrix.prod

band_hic_matrix.**prod**(*axis=None*)

Compute the product of the values in the matrix or along a given axis.

**Parameters**

**axis** (*None, int, or {'row','col','diag'}, optional*) – Axis along which to compute the product:

- None: product over all stored values (and default for missing).
- 0 or 'row': per-row reduction.
- 1 or 'col': per-column reduction.
- 'diag': per-diagonal reduction.

Default is None.

**Returns**

Product(s) along the specified axis.

**Return type**

scalar or ndarray

**Raises**

**ValueError** – If axis is not one of the supported values.

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(1, 10).reshape(3,3), diag_num=2)
>>> mat.prod()  # product of all elements
0
>>> mat.prod(axis='row')
array([ 0, 60, 0])
```

#### 3.1.4.8 bandhic.band_hic_matrix.ptp

band_hic_matrix.**ptp**(*axis=None*)

Compute the peak-to-peak (maximum - minimum) value of the matrix or along a given axis.

**Parameters**

**axis** (*None, int, or {'row','col','diag'}, optional*) – Axis along which to compute the peak-to-peak value:

- None: ptp over all stored values (and default for missing).
- 0 or 'row': per-row reduction.
- 1 or 'col': per-column reduction.
- 'diag': per-diagonal reduction.

Default is None.

**Returns**
Peak-to-peak value(s) along the specified axis.

**Return type**
scalar or ndarray

**Raises**
**ValueError** – If axis is not one of the supported values.

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(9).reshape(3,3), diag_num=2)
>>> mat.ptp()  # ptp of all elements
8
>>> mat.ptp(axis='row')
array([1, 4, 8])
```

#### 3.1.4.9 bandhic.band_hic_matrix.all

band_hic_matrix.**all**(*axis=None*, *banded_only=False*)

Test whether all (or any) array elements along a given axis evaluate to True.

**Parameters**

- **axis** (*None, int, or {'row','col','diag'}, optional*) – Axis along which to test.
  - None: test all stored values (and default for missing).
  - 0 or 'row': per-row reduction.
  - 1 or 'col': per-column reduction.
  - 'diag': per-diagonal reduction.
- **banded_only** (*bool, optional*) – If True, only consider stored band elements; ignore out-of-band values. Default is False.

**Returns**
Boolean result(s) of the test.

**Return type**
bool or ndarray

**Raises**
**ValueError** – If axis is not supported.

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(3), diag_num=2)
>>> mat.all()
False
>>> mat.any(axis='diag', banded_only=True)
array([ True, False])
```

#### 3.1.4.10 bandhic.band_hic_matrix.any

band_hic_matrix.**any**(*axis=None*, *banded_only=False*)

Test whether all (or any) array elements along a given axis evaluate to True.

**Parameters**

- **axis** (*None, int, or {'row','col','diag'}, optional*) – Axis along which to test:
  - None: test all stored values (and default for missing).
  - 0 or 'row': per-row reduction.
  - 1 or 'col': per-column reduction.
  - 'diag': per-diagonal reduction.
- **banded_only** (*bool, optional*) – If True, only consider stored band elements; ignore out-of-band values. Default is False.

**Returns**

Boolean result(s) of the test.

**Return type**

bool or ndarray

**Raises**

**ValueError** – If axis is not supported.

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(3), diag_num=2)
>>> mat.all()
False
>>> mat.any(axis='diag', banded_only=True)
array([ True, False])
```

#### 3.1.4.11 bandhic.band_hic_matrix.normalize

band_hic_matrix.**normalize**(*inplace=False*)

Normalize each diagonal of the matrix to have zero mean and unit variance. This modifies the matrix in place. :raises UserWarning: If any diagonal has zero standard deviation, it will be set to zero.

**Return type**

None

**Parameters**

**inplace** (*bool*)

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.arange(9).reshape(3,3), diag_num=2)
>>> mat = mat.normalize()
```

### 3.1.5 Vectorized Computation Methods

These methods allow for the application of universal functions (ufuncs) to the matrix, enabling element-wise operations similar to those in NumPy.

| | |
|---|---|
| band_hic_matrix.absolute(self, *args, **kwargs) | Perform element-wise 'absolute' operation. |
| band_hic_matrix.add(self, other, *args, **kwargs) | Perform element-wise 'add' operation with two inputs. |
| band_hic_matrix.arccos(self, *args, **kwargs) | Perform element-wise 'arccos' operation. |
| band_hic_matrix.arccosh(self, *args, **kwargs) | Perform element-wise 'arccosh' operation. |
| band_hic_matrix.arcsin(self, *args, **kwargs) | Perform element-wise 'arcsin' operation. |
| band_hic_matrix.arcsinh(self, *args, **kwargs) | Perform element-wise 'arcsinh' operation. |
| band_hic_matrix.arctan(self, *args, **kwargs) | Perform element-wise 'arctan' operation. |
| band_hic_matrix.arctan2(self, other, *args, ...) | Perform element-wise 'arctan2' operation with two inputs. |
| band_hic_matrix.arctanh(self, *args, **kwargs) | Perform element-wise 'arctanh' operation. |
| band_hic_matrix.bitwise_and(self, other, ...) | Perform element-wise 'bitwise_and' operation with two inputs. |
| band_hic_matrix.bitwise_or(self, other, ...) | Perform element-wise 'bitwise_or' operation with two inputs. |
| band_hic_matrix.bitwise_xor(self, other, ...) | Perform element-wise 'bitwise_xor' operation with two inputs. |
| band_hic_matrix.cbrt(self, *args, **kwargs) | Perform element-wise 'cbrt' operation. |
| band_hic_matrix.conj(self, *args, **kwargs) | Perform element-wise 'conj' operation. |
| band_hic_matrix.conjugate(self, *args, **kwargs) | Perform element-wise 'conjugate' operation. |
| band_hic_matrix.cos(self, *args, **kwargs) | Perform element-wise 'cos' operation. |
| band_hic_matrix.cosh(self, *args, **kwargs) | Perform element-wise 'cosh' operation. |
| band_hic_matrix.deg2rad(self, *args, **kwargs) | Perform element-wise 'deg2rad' operation. |
| band_hic_matrix.degrees(self, *args, **kwargs) | Perform element-wise 'degrees' operation. |
| band_hic_matrix.divide(self, other, *args, ...) | Perform element-wise 'divide' operation with two inputs. |
| band_hic_matrix.divmod(self, other, *args, ...) | Perform element-wise 'divmod' operation with two inputs. |
| band_hic_matrix.equal(self, other, *args, ...) | Perform element-wise 'equal' operation with two inputs. |
| band_hic_matrix.exp(self, *args, **kwargs) | Perform element-wise 'exp' operation. |
| band_hic_matrix.exp2(self, *args, **kwargs) | Perform element-wise 'exp2' operation. |
| band_hic_matrix.expm1(self, *args, **kwargs) | Perform element-wise 'expm1' operation. |
| band_hic_matrix.fabs(self, *args, **kwargs) | Perform element-wise 'fabs' operation. |
| band_hic_matrix.float_power(self, other, ...) | Perform element-wise 'float_power' operation with two inputs. |
| band_hic_matrix.floor_divide(self, other, ...) | Perform element-wise 'floor_divide' operation with two inputs. |
| band_hic_matrix.fmod(self, other, *args, ...) | Perform element-wise 'fmod' operation with two inputs. |
| band_hic_matrix.gcd(self, other, *args, **kwargs) | Perform element-wise 'gcd' operation with two inputs. |
| band_hic_matrix.greater(self, other, *args, ...) | Perform element-wise 'greater' operation with two inputs. |
| band_hic_matrix.greater_equal(self, other, ...) | Perform element-wise 'greater_equal' operation with two inputs. |
| band_hic_matrix.heaviside(self, other, ...) | Perform element-wise 'heaviside' operation with two inputs. |
| band_hic_matrix.hypot(self, other, *args, ...) | Perform element-wise 'hypot' operation with two inputs. |
| band_hic_matrix.invert(self, *args, **kwargs) | Perform element-wise 'invert' operation. |
| band_hic_matrix.lcm(self, other, *args, **kwargs) | Perform element-wise 'lcm' operation with two inputs. |
| band_hic_matrix.left_shift(self, other, ...) | Perform element-wise 'left_shift' operation with two inputs. |

continues on next page

Table 4 – continued from previous page

| | |
|---|---|
| band_hic_matrix.less(self, other, *args, ...) | Perform element-wise 'less' operation with two inputs. |
| band_hic_matrix.less_equal(self, other, ...) | Perform element-wise 'less_equal' operation with two inputs. |
| band_hic_matrix.log(self, *args, **kwargs) | Perform element-wise 'log' operation. |
| band_hic_matrix.log1p(self, *args, **kwargs) | Perform element-wise 'log1p' operation. |
| band_hic_matrix.log2(self, *args, **kwargs) | Perform element-wise 'log2' operation. |
| band_hic_matrix.log10(self, *args, **kwargs) | Perform element-wise 'log10' operation. |
| band_hic_matrix.logaddexp(self, other, ...) | Perform element-wise 'logaddexp' operation with two inputs. |
| band_hic_matrix.logaddexp2(self, other, ...) | Perform element-wise 'logaddexp2' operation with two inputs. |
| band_hic_matrix.logical_and(self, other, ...) | Perform element-wise 'logical_and' operation with two inputs. |
| band_hic_matrix.logical_or(self, other, ...) | Perform element-wise 'logical_or' operation with two inputs. |
| band_hic_matrix.logical_xor(self, other, ...) | Perform element-wise 'logical_xor' operation with two inputs. |
| band_hic_matrix.maximum(self, other, *args, ...) | Perform element-wise 'maximum' operation with two inputs. |
| band_hic_matrix.minimum(self, other, *args, ...) | Perform element-wise 'minimum' operation with two inputs. |
| band_hic_matrix.mod(self, other, *args, **kwargs) | Perform element-wise 'mod' operation with two inputs. |
| band_hic_matrix.multiply(self, other, *args, ...) | Perform element-wise 'multiply' operation with two inputs. |
| band_hic_matrix.negative(self, *args, **kwargs) | Perform element-wise 'negative' operation. |
| band_hic_matrix.not_equal(self, other, ...) | Perform element-wise 'not_equal' operation with two inputs. |
| band_hic_matrix.positive(self, *args, **kwargs) | Perform element-wise 'positive' operation. |
| band_hic_matrix.power(self, other, *args, ...) | Perform element-wise 'power' operation with two inputs. |
| band_hic_matrix.rad2deg(self, *args, **kwargs) | Perform element-wise 'rad2deg' operation. |
| band_hic_matrix.radians(self, *args, **kwargs) | Perform element-wise 'radians' operation. |
| band_hic_matrix.reciprocal(self, *args, **kwargs) | Perform element-wise 'reciprocal' operation. |
| band_hic_matrix.remainder(self, other, ...) | Perform element-wise 'remainder' operation with two inputs. |
| band_hic_matrix.right_shift(self, other, ...) | Perform element-wise 'right_shift' operation with two inputs. |
| band_hic_matrix.rint(self, *args, **kwargs) | Perform element-wise 'rint' operation. |
| band_hic_matrix.sign(self, *args, **kwargs) | Perform element-wise 'sign' operation. |
| band_hic_matrix.sin(self, *args, **kwargs) | Perform element-wise 'sin' operation. |
| band_hic_matrix.sinh(self, *args, **kwargs) | Perform element-wise 'sinh' operation. |
| band_hic_matrix.sqrt(self, *args, **kwargs) | Perform element-wise 'sqrt' operation. |
| band_hic_matrix.square(self, *args, **kwargs) | Perform element-wise 'square' operation. |
| band_hic_matrix.subtract(self, other, *args, ...) | Perform element-wise 'subtract' operation with two inputs. |
| band_hic_matrix.tan(self, *args, **kwargs) | Perform element-wise 'tan' operation. |
| band_hic_matrix.tanh(self, *args, **kwargs) | Perform element-wise 'tanh' operation. |
| band_hic_matrix.true_divide(self, other, ...) | Perform element-wise 'true_divide' operation with two inputs. |

#### 3.1.5.1 bandhic.band_hic_matrix.absolute

band_hic_matrix.**absolute**(*self*, **args*, ***kwargs*)

Perform element-wise 'absolute' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.absolute.
- ****kwargs** (*dict*) – Keyword arguments for numpy.absolute.

**Returns**

Result of element-wise 'absolute' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.absolute

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.absolute()
```

#### 3.1.5.2 bandhic.band_hic_matrix.add

band_hic_matrix.**add**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'add' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.add.
- ****kwargs** (*dict*) – Keyword arguments for numpy.add.

**Returns**

Result of element-wise 'add' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.add

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.add(other)
```

#### 3.1.5.3 bandhic.band_hic_matrix.arccos

band_hic_matrix.**arccos**(*self*, **args*, ***kwargs*)

Perform element-wise 'arccos' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.arccos.
- ****kwargs** (*dict*) – Keyword arguments for numpy.arccos.

**Returns**

Result of element-wise 'arccos' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.arccos

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.arccos()
```

#### 3.1.5.4 bandhic.band_hic_matrix.arccosh

band_hic_matrix.**arccosh**(*self*, **args*, ***kwargs*)

Perform element-wise 'arccosh' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.arccosh.
- ****kwargs** (*dict*) – Keyword arguments for numpy.arccosh.

**Returns**

Result of element-wise 'arccosh' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.arccosh

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.arccosh()
```

#### 3.1.5.5 bandhic.band_hic_matrix.arcsin

band_hic_matrix.**arcsin**(*self*, **args*, ***kwargs*)

Perform element-wise 'arcsin' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.arcsin.
- ****kwargs** (*dict*) – Keyword arguments for numpy.arcsin.

**Returns**

Result of element-wise 'arcsin' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.arcsin

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.arcsin()
```

#### 3.1.5.6 bandhic.band_hic_matrix.arcsinh

band_hic_matrix.**arcsinh**(*self*, **args*, ***kwargs*)

Perform element-wise 'arcsinh' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.arcsinh.
- ****kwargs** (*dict*) – Keyword arguments for numpy.arcsinh.

**Returns**

Result of element-wise 'arcsinh' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.arcsinh

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.arcsinh()
```

#### 3.1.5.7 bandhic.band_hic_matrix.arctan

band_hic_matrix.**arctan**(*self*, **args*, ***kwargs*)

Perform element-wise 'arctan' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.arctan.
- ****kwargs** (*dict*) – Keyword arguments for numpy.arctan.

**Returns**

Result of element-wise 'arctan' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.arctan

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.arctan()
```

#### 3.1.5.8 bandhic.band_hic_matrix.arctan2

band_hic_matrix.**arctan2**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'arctan2' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.arctan2.
- ****kwargs** (*dict*) – Keyword arguments for numpy.arctan2.

**Returns**

Result of element-wise 'arctan2' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.arctan2

#### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.arctan2(other)
```

#### 3.1.5.9 bandhic.band_hic_matrix.arctanh

band_hic_matrix.**arctanh**(*self*, **args*, ***kwargs*)

Perform element-wise 'arctanh' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.arctanh.
- ****kwargs** (*dict*) – Keyword arguments for numpy.arctanh.

**Returns**

Result of element-wise 'arctanh' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.arctanh

#### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.arctanh()
```

#### 3.1.5.10 bandhic.band_hic_matrix.bitwise_and

band_hic_matrix.**bitwise_and**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'bitwise_and' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.bitwise_and.
- ****kwargs** (*dict*) – Keyword arguments for numpy.bitwise_and.

**Returns**

Result of element-wise 'bitwise_and' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.bitwise_and

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.bitwise_and(other)
```

#### 3.1.5.11 bandhic.band_hic_matrix.bitwise_or

band_hic_matrix.**bitwise_or**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'bitwise_or' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.bitwise_or.
- ****kwargs** (*dict*) – Keyword arguments for numpy.bitwise_or.

**Returns**
Result of element-wise 'bitwise_or' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.bitwise_or

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.bitwise_or(other)
```

#### 3.1.5.12 bandhic.band_hic_matrix.bitwise_xor

band_hic_matrix.**bitwise_xor**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'bitwise_xor' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.

- ***args** (*tuple*) – Additional positional arguments for numpy.bitwise_xor.
- ****kwargs** (*dict*) – Keyword arguments for numpy.bitwise_xor.

**Returns**
Result of element-wise 'bitwise_xor' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.bitwise_xor

#### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.bitwise_xor(other)
```

### 3.1.5.13 bandhic.band_hic_matrix.cbrt

band_hic_matrix.**cbrt**(*self*, **args*, ***kwargs*)

Perform element-wise 'cbrt' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.cbrt.
- ****kwargs** (*dict*) – Keyword arguments for numpy.cbrt.

**Returns**
Result of element-wise 'cbrt' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.cbrt

#### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.cbrt()
```

### 3.1.5.14 bandhic.band_hic_matrix.conj

band_hic_matrix.**conj**(*self*, **args*, ***kwargs*)

Perform element-wise 'conj' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.conj.
- ****kwargs** (*dict*) – Keyword arguments for numpy.conj.

**Returns**
Result of element-wise 'conj' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.conj

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.conj()
```

#### 3.1.5.15 bandhic.band_hic_matrix.conjugate

band_hic_matrix.**conjugate**(*self*, **args*, ***kwargs*)

Perform element-wise 'conjugate' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.conjugate.
- ****kwargs** (*dict*) – Keyword arguments for numpy.conjugate.

**Returns**
Result of element-wise 'conjugate' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.conjugate

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.conjugate()
```

#### 3.1.5.16 bandhic.band_hic_matrix.cos

band_hic_matrix.**cos**(*self*, **args*, ***kwargs*)

Perform element-wise 'cos' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.cos.
- ****kwargs** (*dict*) – Keyword arguments for numpy.cos.

**Returns**
Result of element-wise 'cos' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.cos

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.cos()
```

#### 3.1.5.17 bandhic.band_hic_matrix.cosh

band_hic_matrix.**cosh**(*self*, **args*, ***kwargs*)

Perform element-wise 'cosh' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.cosh.
- ****kwargs** (*dict*) – Keyword arguments for numpy.cosh.

**Returns**
Result of element-wise 'cosh' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.cosh

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.cosh()
```

#### 3.1.5.18 bandhic.band_hic_matrix.deg2rad

band_hic_matrix.**deg2rad**(*self*, **args*, ***kwargs*)

Perform element-wise 'deg2rad' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.deg2rad.
- ****kwargs** (*dict*) – Keyword arguments for numpy.deg2rad.

**Returns**

Result of element-wise 'deg2rad' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.deg2rad

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.deg2rad()
```

#### 3.1.5.19 bandhic.band_hic_matrix.degrees

band_hic_matrix.**degrees**(*self*, **args*, ***kwargs*)

Perform element-wise 'degrees' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.degrees.
- ****kwargs** (*dict*) – Keyword arguments for numpy.degrees.

**Returns**

Result of element-wise 'degrees' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.degrees

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.degrees()
```

#### 3.1.5.20 bandhic.band_hic_matrix.divide

band_hic_matrix.**divide**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'divide' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.divide.
- ****kwargs** (*dict*) – Keyword arguments for numpy.divide.

**Returns**

Result of element-wise 'divide' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.divide

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.divide(other)
```

#### 3.1.5.21 bandhic.band_hic_matrix.divmod

band_hic_matrix.**divmod**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'divmod' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.divmod.
- ****kwargs** (*dict*) – Keyword arguments for numpy.divmod.

**Returns**

Result of element-wise 'divmod' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.divmod

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.divmod(other)
```

#### 3.1.5.22 bandhic.band_hic_matrix.equal

band_hic_matrix.**equal**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'equal' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.equal.
- ****kwargs** (*dict*) – Keyword arguments for numpy.equal.

**Returns**

Result of element-wise 'equal' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.equal

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.equal(other)
```

#### 3.1.5.23 bandhic.band_hic_matrix.exp

band_hic_matrix.**exp**(*self*, **args*, ***kwargs*)

Perform element-wise 'exp' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.exp.
- ****kwargs** (*dict*) – Keyword arguments for numpy.exp.

**Returns**

Result of element-wise 'exp' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.exp

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.exp()
```

#### 3.1.5.24 bandhic.band_hic_matrix.exp2

band_hic_matrix.**exp2**(*self*, **args*, ***kwargs*)

Perform element-wise 'exp2' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.exp2.
- ****kwargs** (*dict*) – Keyword arguments for numpy.exp2.

**Returns**

Result of element-wise 'exp2' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.exp2

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.exp2()
```

#### 3.1.5.25 bandhic.band_hic_matrix.expm1

band_hic_matrix.**expm1**(*self*, **args*, ***kwargs*)

Perform element-wise 'expm1' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.expm1.
- ****kwargs** (*dict*) – Keyword arguments for numpy.expm1.

**Returns**

Result of element-wise 'expm1' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.expm1

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.expm1()
```

#### 3.1.5.26 bandhic.band_hic_matrix.fabs

band_hic_matrix.**fabs**(*self*, **args*, ***kwargs*)

Perform element-wise 'fabs' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.fabs.
- ****kwargs** (*dict*) – Keyword arguments for numpy.fabs.

**Returns**

Result of element-wise 'fabs' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.fabs

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.fabs()
```

#### 3.1.5.27 bandhic.band_hic_matrix.float_power

band_hic_matrix.**float_power**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'float_power' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.float_power.
- ****kwargs** (*dict*) – Keyword arguments for numpy.float_power.

**Returns**

Result of element-wise 'float_power' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.float_power

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.float_power(other)
```

#### 3.1.5.28 bandhic.band_hic_matrix.floor_divide

band_hic_matrix.**floor_divide**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'floor_divide' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.floor_divide.
- ****kwargs** (*dict*) – Keyword arguments for numpy.floor_divide.

**Returns**
Result of element-wise 'floor_divide' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.floor_divide

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.floor_divide(other)
```

#### 3.1.5.29 bandhic.band_hic_matrix.fmod

band_hic_matrix.**fmod**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'fmod' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.

- ***args** (*tuple*) – Additional positional arguments for numpy.fmod.
- ****kwargs** (*dict*) – Keyword arguments for numpy.fmod.

**Returns**
Result of element-wise 'fmod' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.fmod

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.fmod(other)
```

#### 3.1.5.30 bandhic.band_hic_matrix.gcd

band_hic_matrix.**gcd**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'gcd' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.gcd.
- ****kwargs** (*dict*) – Keyword arguments for numpy.gcd.

**Returns**
Result of element-wise 'gcd' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.gcd

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.gcd(other)
```

#### 3.1.5.31 bandhic.band_hic_matrix.greater

band_hic_matrix.**greater**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'greater' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.greater.
- ****kwargs** (*dict*) – Keyword arguments for numpy.greater.

**Returns**

Result of element-wise 'greater' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.greater

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.greater(other)
```

#### 3.1.5.32 bandhic.band_hic_matrix.greater_equal

band_hic_matrix.**greater_equal**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'greater_equal' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.greater_equal.
- ****kwargs** (*dict*) – Keyword arguments for numpy.greater_equal.

**Returns**

Result of element-wise 'greater_equal' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.greater_equal

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.greater_equal(other)
```

#### 3.1.5.33 bandhic.band_hic_matrix.heaviside

band_hic_matrix.**heaviside**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'heaviside' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.heaviside.
- ****kwargs** (*dict*) – Keyword arguments for numpy.heaviside.

**Returns**

Result of element-wise 'heaviside' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.heaviside

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.heaviside(other)
```

#### 3.1.5.34 bandhic.band_hic_matrix.hypot

band_hic_matrix.**hypot**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'hypot' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.hypot.
- ****kwargs** (*dict*) – Keyword arguments for numpy.hypot.

**Returns**

Result of element-wise 'hypot' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.hypot

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.hypot(other)
```

#### 3.1.5.35 bandhic.band_hic_matrix.invert

band_hic_matrix.**invert**(*self*, **args*, ***kwargs*)

Perform element-wise 'invert' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.invert.
- ****kwargs** (*dict*) – Keyword arguments for numpy.invert.

**Returns**

Result of element-wise 'invert' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.invert

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.invert()
```

#### 3.1.5.36 bandhic.band_hic_matrix.lcm

band_hic_matrix.**lcm**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'lcm' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.lcm.
- ****kwargs** (*dict*) – Keyword arguments for numpy.lcm.

**Returns**

Result of element-wise 'lcm' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.lcm

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.lcm(other)
```

#### 3.1.5.37 bandhic.band_hic_matrix.left_shift

band_hic_matrix.**left_shift**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'left_shift' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.left_shift.
- ****kwargs** (*dict*) – Keyword arguments for numpy.left_shift.

**Returns**
Result of element-wise 'left_shift' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.left_shift

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.left_shift(other)
```

#### 3.1.5.38 bandhic.band_hic_matrix.less

band_hic_matrix.**less**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'less' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.

- ***args** (*tuple*) – Additional positional arguments for numpy.less.
- ****kwargs** (*dict*) – Keyword arguments for numpy.less.

**Returns**
Result of element-wise 'less' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.less

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.less(other)
```

#### 3.1.5.39 bandhic.band_hic_matrix.less_equal

band_hic_matrix.**less_equal**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'less_equal' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.less_equal.
- ****kwargs** (*dict*) – Keyword arguments for numpy.less_equal.

**Returns**
Result of element-wise 'less_equal' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.less_equal

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.less_equal(other)
```

#### 3.1.5.40 bandhic.band_hic_matrix.log

band_hic_matrix.**log**(*self*, **args*, ***kwargs*)

Perform element-wise 'log' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.log.
- ****kwargs** (*dict*) – Keyword arguments for numpy.log.

**Returns**

Result of element-wise 'log' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.log

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.log()
```

#### 3.1.5.41 bandhic.band_hic_matrix.log1p

band_hic_matrix.**log1p**(*self*, **args*, ***kwargs*)

Perform element-wise 'log1p' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.log1p.
- ****kwargs** (*dict*) – Keyword arguments for numpy.log1p.

**Returns**

Result of element-wise 'log1p' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.log1p

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.log1p()
```

#### 3.1.5.42 bandhic.band_hic_matrix.log2

band_hic_matrix.**log2**(*self*, **args*, ***kwargs*)

Perform element-wise 'log2' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.log2.
- ****kwargs** (*dict*) – Keyword arguments for numpy.log2.

**Returns**

Result of element-wise 'log2' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.log2

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.log2()
```

#### 3.1.5.43 bandhic.band_hic_matrix.log10

band_hic_matrix.**log10**(*self*, **args*, ***kwargs*)

Perform element-wise 'log10' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.log10.
- ****kwargs** (*dict*) – Keyword arguments for numpy.log10.

**Returns**

Result of element-wise 'log10' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.log10

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.log10()
```

#### 3.1.5.44 bandhic.band_hic_matrix.logaddexp

band_hic_matrix.**logaddexp**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'logaddexp' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.logaddexp.
- ****kwargs** (*dict*) – Keyword arguments for numpy.logaddexp.

**Returns**

Result of element-wise 'logaddexp' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.logaddexp

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.logaddexp(other)
```

#### 3.1.5.45 bandhic.band_hic_matrix.logaddexp2

band_hic_matrix.**logaddexp2**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'logaddexp2' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.logaddexp2.
- ****kwargs** (*dict*) – Keyword arguments for numpy.logaddexp2.

**Returns**

Result of element-wise 'logaddexp2' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.logaddexp2

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.logaddexp2(other)
```

#### 3.1.5.46 bandhic.band_hic_matrix.logical_and

band_hic_matrix.**logical_and**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'logical_and' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.logical_and.
- ****kwargs** (*dict*) – Keyword arguments for numpy.logical_and.

**Returns**

Result of element-wise 'logical_and' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.logical_and

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.logical_and(other)
```

#### 3.1.5.47 bandhic.band_hic_matrix.logical_or

band_hic_matrix.**logical_or**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'logical_or' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.logical_or.
- ****kwargs** (*dict*) – Keyword arguments for numpy.logical_or.

**Returns**

Result of element-wise 'logical_or' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.logical_or

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.logical_or(other)
```

#### 3.1.5.48 bandhic.band_hic_matrix.logical_xor

band_hic_matrix.**logical_xor**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'logical_xor' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.logical_xor.
- ****kwargs** (*dict*) – Keyword arguments for numpy.logical_xor.

**Returns**

Result of element-wise 'logical_xor' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.logical_xor

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.logical_xor(other)
```

#### 3.1.5.49 bandhic.band_hic_matrix.maximum

band_hic_matrix.**maximum**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'maximum' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.maximum.
- ****kwargs** (*dict*) – Keyword arguments for numpy.maximum.

**Returns**
Result of element-wise 'maximum' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.maximum

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.maximum(other)
```

#### 3.1.5.50 bandhic.band_hic_matrix.minimum

band_hic_matrix.**minimum**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'minimum' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.minimum.
- ****kwargs** (*dict*) – Keyword arguments for numpy.minimum.

**Returns**
Result of element-wise 'minimum' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.minimum

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.minimum(other)
```

#### 3.1.5.51 bandhic.band_hic_matrix.mod

band_hic_matrix.**mod**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'mod' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.mod.
- ****kwargs** (*dict*) – Keyword arguments for numpy.mod.

**Returns**
Result of element-wise 'mod' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.mod

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.mod(other)
```

#### 3.1.5.52 bandhic.band_hic_matrix.multiply

band_hic_matrix.**multiply**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'multiply' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.multiply.
- ****kwargs** (*dict*) – Keyword arguments for numpy.multiply.

**Returns**
Result of element-wise 'multiply' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.multiply

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.multiply(other)
```

#### 3.1.5.53 bandhic.band_hic_matrix.negative

band_hic_matrix.**negative**(*self*, **args*, ***kwargs*)

Perform element-wise 'negative' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.negative.
- ****kwargs** (*dict*) – Keyword arguments for numpy.negative.

**Returns**

Result of element-wise 'negative' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.negative

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.negative()
```

#### 3.1.5.54 bandhic.band_hic_matrix.not_equal

band_hic_matrix.**not_equal**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'not_equal' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.not_equal.
- ****kwargs** (*dict*) – Keyword arguments for numpy.not_equal.

**Returns**

Result of element-wise 'not_equal' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.not_equal

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.not_equal(other)
```

#### 3.1.5.55 bandhic.band_hic_matrix.positive

band_hic_matrix.**positive**(*self*, **args*, ***kwargs*)

Perform element-wise 'positive' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.positive.
- ****kwargs** (*dict*) – Keyword arguments for numpy.positive.

**Returns**

Result of element-wise 'positive' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.positive

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.positive()
```

#### 3.1.5.56 bandhic.band_hic_matrix.power

band_hic_matrix.**power**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'power' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.power.
- ****kwargs** (*dict*) – Keyword arguments for numpy.power.

**Returns**

Result of element-wise 'power' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.power

#### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.power(other)
```

#### 3.1.5.57 bandhic.band_hic_matrix.rad2deg

band_hic_matrix.**rad2deg**(*self*, **args*, ***kwargs*)

Perform element-wise 'rad2deg' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.rad2deg.
- ****kwargs** (*dict*) – Keyword arguments for numpy.rad2deg.

**Returns**

Result of element-wise 'rad2deg' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.rad2deg

#### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.rad2deg()
```

#### 3.1.5.58 bandhic.band_hic_matrix.radians

band_hic_matrix.**radians**(*self*, **args*, ***kwargs*)

Perform element-wise 'radians' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.radians.
- ****kwargs** (*dict*) – Keyword arguments for numpy.radians.

**Returns**

Result of element-wise 'radians' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.radians

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.radians()
```

#### 3.1.5.59 bandhic.band_hic_matrix.reciprocal

band_hic_matrix.**reciprocal**(*self*, **args*, ***kwargs*)

Perform element-wise 'reciprocal' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.reciprocal.
- ****kwargs** (*dict*) – Keyword arguments for numpy.reciprocal.

**Returns**
Result of element-wise 'reciprocal' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.reciprocal

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.reciprocal()
```

#### 3.1.5.60 bandhic.band_hic_matrix.remainder

band_hic_matrix.**remainder**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'remainder' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.remainder.
- ****kwargs** (*dict*) – Keyword arguments for numpy.remainder.

**Returns**
Result of element-wise 'remainder' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.remainder

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.remainder(other)
```

#### 3.1.5.61 bandhic.band_hic_matrix.right_shift

band_hic_matrix.**right_shift**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'right_shift' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.right_shift.
- ****kwargs** (*dict*) – Keyword arguments for numpy.right_shift.

**Returns**
Result of element-wise 'right_shift' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.right_shift

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.right_shift(other)
```

#### 3.1.5.62 bandhic.band_hic_matrix.rint

band_hic_matrix.**rint**(*self*, **args*, ***kwargs*)

Perform element-wise 'rint' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.rint.
- ****kwargs** (*dict*) – Keyword arguments for numpy.rint.

**Returns**
Result of element-wise 'rint' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.rint

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.rint()
```

#### 3.1.5.63 bandhic.band_hic_matrix.sign

band_hic_matrix.**sign**(*self*, **args*, ***kwargs*)

Perform element-wise 'sign' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.sign.
- ****kwargs** (*dict*) – Keyword arguments for numpy.sign.

**Returns**
Result of element-wise 'sign' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.sign

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.sign()
```

#### 3.1.5.64 bandhic.band_hic_matrix.sin

band_hic_matrix.**sin**(*self*, **args*, ***kwargs*)

Perform element-wise 'sin' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.sin.
- ****kwargs** (*dict*) – Keyword arguments for numpy.sin.

**Returns**
Result of element-wise 'sin' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.sin

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.sin()
```

#### 3.1.5.65 bandhic.band_hic_matrix.sinh

band_hic_matrix.**sinh**(*self*, **args*, ***kwargs*)

Perform element-wise 'sinh' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.sinh.
- ****kwargs** (*dict*) – Keyword arguments for numpy.sinh.

**Returns**
Result of element-wise 'sinh' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.sinh

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.sinh()
```

#### 3.1.5.66 bandhic.band_hic_matrix.sqrt

band_hic_matrix.**sqrt**(*self*, **args*, ***kwargs*)

Perform element-wise 'sqrt' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.sqrt.
- ****kwargs** (*dict*) – Keyword arguments for numpy.sqrt.

**Returns**

Result of element-wise 'sqrt' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.sqrt

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.sqrt()
```

#### 3.1.5.67 bandhic.band_hic_matrix.square

band_hic_matrix.**square**(*self*, **args*, ***kwargs*)

Perform element-wise 'square' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.square.
- ****kwargs** (*dict*) – Keyword arguments for numpy.square.

**Returns**

Result of element-wise 'square' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.square

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.square()
```

#### 3.1.5.68 bandhic.band_hic_matrix.subtract

band_hic_matrix.**subtract**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'subtract' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.subtract.
- ****kwargs** (*dict*) – Keyword arguments for numpy.subtract.

**Returns**

Result of element-wise 'subtract' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.subtract

**Examples**

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.subtract(other)
```

#### 3.1.5.69 bandhic.band_hic_matrix.tan

band_hic_matrix.**tan**(*self*, **args*, ***kwargs*)

Perform element-wise 'tan' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.tan.
- ****kwargs** (*dict*) – Keyword arguments for numpy.tan.

**Returns**

Result of element-wise 'tan' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.tan

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.tan()
```

#### 3.1.5.70 bandhic.band_hic_matrix.tanh

band_hic_matrix.**tanh**(*self*, **args*, ***kwargs*)

Perform element-wise 'tanh' operation.

**Parameters**

- **self** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.tanh.
- ****kwargs** (*dict*) – Keyword arguments for numpy.tanh.

**Returns**

Result of element-wise 'tanh' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.tanh

### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = mat.tanh()
```

#### 3.1.5.71 bandhic.band_hic_matrix.true_divide

band_hic_matrix.**true_divide**(*self*, *other*, **args*, ***kwargs*)

Perform element-wise 'true_divide' operation with two inputs.

**Parameters**

- **self** (band_hic_matrix) – First input matrix.
- **other** (band_hic_matrix *or array-like*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.true_divide.
- ****kwargs** (*dict*) – Keyword arguments for numpy.true_divide.

**Returns**

Result of element-wise 'true_divide' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.true_divide

#### Examples

```
>>> from bandhic import band_hic_matrix
>>> mat = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> other = mat.copy()
>>> result = mat.true_divide(other)
```

### 3.1.6 Data Type Conversion and Representation

This section includes methods for converting the matrix to different data types or representations, such as dense, sparse, or masked arrays.

| | |
|---|---|
| *band_hic_matrix.todense*() | Convert the band matrix to a dense format. |
| *band_hic_matrix.tocoo*([drop_zeros]) | Convert the matrix to COO format. |
| *band_hic_matrix.tocsr*() | Convert the matrix to CSR format. |
| *band_hic_matrix.copy*() | Deep copy the object. |
| *band_hic_matrix.astype*(dtype[, copy]) | Cast data to new dtype. |
| *band_hic_matrix.__repr__*() | Return a string representation of the band_hic_matrix object. |
| *band_hic_matrix.__str__*() | Return a string representation of the band_hic_matrix object. |
| *band_hic_matrix.__array__*([copy]) | Return the data as a NumPy array. |
| *band_hic_matrix.__array_priority__*() | Return the priority for array operations. |

#### 3.1.6.1 bandhic.band_hic_matrix.todense

band_hic_matrix.**todense**()

Convert the band matrix to a dense format.

**Returns**

The dense (square) matrix. Masked if mask is set.

**Return type**

ndarray or MaskedArray

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(3), diag_num=2)
>>> dense = mat.todense()
>>> dense.shape
(3, 3)
```

#### 3.1.6.2 bandhic.band_hic_matrix.tocoo

band_hic_matrix.**tocoo**(*drop_zeros=True*)

Convert the matrix to COO format.

**Parameters**
**drop_zeros** (*bool, optional*) – If True, zero entries will be dropped from the COO format. Default is True.

**Returns**
The matrix in scipy COO sparse format.

**Return type**
coo_array

### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> coo = mat.tocoo()
>>> coo.shape
(3, 3)
```

#### 3.1.6.3 bandhic.band_hic_matrix.tocsr

band_hic_matrix.**tocsr**()

Convert the matrix to CSR format.

**Returns**
The matrix in scipy CSR sparse format.

**Return type**
csr_array

### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> csr = mat.tocsr()
>>> csr.shape
(3, 3)
```

#### 3.1.6.4 bandhic.band_hic_matrix.copy

band_hic_matrix.**copy**()

Deep copy the object.

**Returns**
A deep copy.

**Return type**
*band_hic_matrix*

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> mat2 = mat.copy()
```

### 3.1.6.5 bandhic.band_hic_matrix.astype

band_hic_matrix.**astype**(*dtype*, *copy=False*)

Cast data to new dtype.

**Parameters**

- **type** (*data-type*) – Target dtype.
- **copy** (*bool*) – If True, the operation is performed in place. Default is False.
- **dtype** (*type*)

**Return type**
*band_hic_matrix*

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> mat = mat.astype(np.float32)
```

### 3.1.6.6 bandhic.band_hic_matrix.__repr__

band_hic_matrix.**__repr__**()

Return a string representation of the band_hic_matrix object.

**Returns**
A string representation of the object.

**Return type**
str

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> repr(mat)
    "band_hic_matrix(shape=(3, 3), diag_num=2, dtype=<class 'numpy.float64'>)"
```

### 3.1.6.7 bandhic.band_hic_matrix.__str__

band_hic_matrix.**__str__**()

Return a string representation of the band_hic_matrix object.

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
```

(continues on next page)

(continued from previous page)

```
>>> print(mat)
    band_hic_matrix(shape=(3, 3), diag_num=2, dtype=<class 'numpy.float64'>)
```

#### 3.1.6.8 bandhic.band_hic_matrix.__array__

band_hic_matrix.**__array__**(*copy=False*)

Return the data as a NumPy array.

**Parameters**
**copy** (*bool, optional*) – If True, returns a copy. Default is False.

**Returns**
Underlying band data array.

**Return type**
ndarray

##### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(3), diag_num=2)
>>> arr = np.array(mat)
```

#### 3.1.6.9 bandhic.band_hic_matrix.__array_priority__

band_hic_matrix.**__array_priority__**()

Return the priority for array operations.

**Returns**
Priority value.

**Return type**
int

##### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(3), diag_num=2)
>>> mat.__array_priority__()
100
```

### 3.1.7 Other Methods

This section includes additional methods that do not fall into the categories above but provide other useful operations for *band_hic_matrix*.

| | |
|---|---|
| *band_hic_matrix.memory_usage*() | Compute memory usage of *band_hic_matirx* object. |
| *band_hic_matrix.__len__*() | Return the number of rows in the band_hic_matrix object. |
| *band_hic_matrix.__hash__*() | Return a hash value for the band_hic_matrix object. |
| *band_hic_matrix.__bool__*() | Truth value of the band_hic_matrix, following NumPy semantics. |

continues on next page

Table 6 – continued from previous page

| | |
|---|---|
| *band_hic_matrix.dump*(filename) | Save the band_hic_matrix object to a file. |

#### 3.1.7.1 bandhic.band_hic_matrix.memory_usage

band_hic_matrix.**memory_usage**()

Compute memory usage of *band_hic_matirx* object.

**Returns**
Size in bytes.

**Return type**
int

### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> mat.memory_usage()
772
```

#### 3.1.7.2 bandhic.band_hic_matrix.__len__

band_hic_matrix.**__len__**()

Return the number of rows in the band_hic_matrix object.

**Returns**
Number of rows.

**Return type**
int

### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> len(mat)
3
```

#### 3.1.7.3 bandhic.band_hic_matrix.__hash__

band_hic_matrix.**__hash__**()

Return a hash value for the band_hic_matrix object.

**Returns**
Hash value.

**Return type**
int

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(3), diag_num=2)
>>> hashed=hash(mat)
```

#### 3.1.7.4 bandhic.band_hic_matrix.__bool__

band_hic_matrix.**__bool__**()

Truth value of the band_hic_matrix, following NumPy semantics.

**Returns**
If the matrix has exactly one element (shape (1,1)), returns its truth value; otherwise raises a ValueError.

**Return type**
bool

**Raises**
**ValueError** – If the matrix contains more than one element.

#### 3.1.7.5 bandhic.band_hic_matrix.dump

band_hic_matrix.**dump**(*filename*)

Save the band_hic_matrix object to a file.

**Parameters**
**filename** (*str*) – The name of the file to save the object to.

**Return type**
None

**Examples**

```
>>> import bandhic as bh
>>> mat = bh.ones((3,3), diag_num=2)
>>> mat.dump('myfile.npz')
```

## 3.2 Creation Functions

These functions create BandHiC matrices with specific properties.

| | |
|---|---|
| *bandhic.ones*(shape[, diag_num, dtype]) | Create a band_hic_matrix object filled with ones. |
| *bandhic.zeros*(shape[, diag_num, dtype]) | Create a band_hic_matrix object filled with zeros. |
| *bandhic.eye*(shape[, diag_num, dtype]) | Create a band_hic_matrix object filled as an identity matrix. |
| *bandhic.full*(shape, fill_value[, diag_num, ...]) | Create a band_hic_matrix object filled with a specified value. |
| *bandhic.ones_like*(other[, dtype]) | Create a band_hic_matrix object matching another matrix, filled with ones. |
| *bandhic.zeros_like*(other[, dtype]) | Create a band_hic_matrix object matching another matrix, filled with zeros. |
| *bandhic.eye_like*(other[, dtype]) | Create a band_hic_matrix object matching another matrix, filled as an identity matrix. |
| *bandhic.full_like*(other, fill_value[, dtype]) | Create a band_hic_matrix object matching another matrix, filled with a specified value. |

### 3.2.1 bandhic.ones

bandhic.**ones**(*shape*, *diag_num=1*, *dtype=<class 'numpy.float64'>*)

Create a band_hic_matrix object filled with ones.

**Parameters**

- **shape** (*tuple of int*) – Matrix shape as (bins, bins).
- **diag_num** (*int, optional*) – Number of diagonals to consider. Default is 1.
- **dtype** (*data-type, optional*) – The data type of the matrix. Default is np.float64.

**Returns**

A band_hic_matrix object with all entries filled with ones.

**Return type**

*band_hic_matrix*

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.ones((5, 5), diag_num=3)
>>> print(mat)
band_hic_matrix(shape=(5, 5), diag_num=3, dtype=<class 'numpy.float64'>)
```

### 3.2.2 bandhic.zeros

bandhic.**zeros**(*shape*, *diag_num=1*, *dtype=<class 'numpy.float64'>*)

Create a band_hic_matrix object filled with zeros.

**Parameters**

- **shape** (*tuple of int*) – Matrix shape as (bins, bins).
- **diag_num** (*int, optional*) – Number of diagonals to consider. Default is 1.
- **dtype** (*data-type, optional*) – The data type of the matrix. Default is np.float64.

**Returns**

A band_hic_matrix object with all entries filled with zeros.

**Return type**

*band_hic_matrix*

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.zeros((5, 5), diag_num=3)
>>> print(mat)
band_hic_matrix(shape=(5, 5), diag_num=3, dtype=<class 'numpy.float64'>)
```

### 3.2.3 bandhic.eye

bandhic.**eye**(*shape*, *diag_num=1*, *dtype=<class 'numpy.float64'>*)

Create a band_hic_matrix object filled as an identity matrix.

**Parameters**

- **shape** (*tuple of int*) – Matrix shape as (bins, bins).
- **diag_num** (*int, optional*) – Number of diagonals to consider. Default is 1.
- **dtype** (*data-type, optional*) – The data type of the matrix. Default is np.float64.

**Returns**

A band_hic_matrix object with ones on the main diagonal and zeros elsewhere.

**Return type**

*band_hic_matrix*

#### Examples

```
>>> mat = eye((5, 5), diag_num=3)
>>> print(mat)
band_hic_matrix(shape=(5, 5), diag_num=3, dtype=<class 'numpy.float64'>)
```

### 3.2.4 bandhic.full

bandhic.**full**(*shape*, *fill_value*, *diag_num=1*, *dtype=<class 'numpy.float64'>*)

Create a band_hic_matrix object filled with a specified value.

**Parameters**

- **shape** (*tuple of int*) – Matrix shape as (bins, bins).
- **fill_value** (*scalar*) – Value to fill the matrix with.
- **diag_num** (*int, optional*) – Number of diagonals to consider. Default is 1.
- **dtype** (*data-type, optional*) – The data type of the matrix. Default is np.float64.

**Returns**

A band_hic_matrix object with all entries filled with *fill_value*.

**Return type**

*band_hic_matrix*

#### Examples

```
>>> import bandhic as bh
>>> mat = bh.full((5, 5), fill_value=7, diag_num=3)
>>> print(mat)
band_hic_matrix(shape=(5, 5), diag_num=3, dtype=<class 'numpy.float64'>)
```

### 3.2.5 bandhic.ones_like

bandhic.**ones_like**(*other*, *dtype=None*)

Create a band_hic_matrix object matching another matrix, filled with ones.

**Parameters**

**other** (band_hic_matrix) – Reference matrix.

**Returns**
A band_hic_matrix object matching *other*, filled with ones.

**Return type**
*band_hic_matrix*

#### Examples

```
>>> mat_ref = zeros((4, 4), diag_num=2)
>>> mat = ones_like(mat_ref)
>>> isinstance(mat, band_hic_matrix)
True
```

### 3.2.6 bandhic.zeros_like

bandhic.**zeros_like**(*other*, *dtype=None*)

Create a band_hic_matrix object matching another matrix, filled with zeros.

**Parameters**
**other** (band_hic_matrix) – Reference matrix.

**Returns**
A band_hic_matrix object matching *other*, filled with zeros.

**Return type**
*band_hic_matrix*

#### Examples

```
>>> mat_ref = zeros((4, 4), diag_num=2)
>>> mat = zeros_like(mat_ref)
>>> isinstance(mat, band_hic_matrix)
True
```

### 3.2.7 bandhic.eye_like

bandhic.**eye_like**(*other*, *dtype=None*)

Create a band_hic_matrix object matching another matrix, filled as an identity matrix.

**Parameters**
**other** (band_hic_matrix) – Reference matrix.

**Returns**
A band_hic_matrix object matching *other*, filled as an identity matrix.

**Return type**
*band_hic_matrix*

#### Examples

```
>>> mat_ref = zeros((4, 4), diag_num=2)
>>> mat = eye_like(mat_ref)
>>> isinstance(mat, band_hic_matrix)
True
```

### 3.2.8 bandhic.full_like

bandhic.**full_like**(*other*, *fill_value*, *dtype=None*)

Create a band_hic_matrix object matching another matrix, filled with a specified value.

**Parameters**

- **other** (band_hic_matrix) – Reference matrix.
- **fill_value** (*scalar*) – Value to fill the matrix.

**Returns**

A band_hic_matrix object matching *other*, filled with *fill_value*.

**Return type**

*band_hic_matrix*

#### Examples

```
>>> mat_ref = zeros((4, 4), diag_num=2)
>>> mat = full_like(mat_ref, fill_value=9)
>>> isinstance(mat, band_hic_matrix)
True
```

## 3.3 Input/Output Functions

These functions handle input and output operations for BandHiC matrices, including saving and loading from various formats.

| | |
|---|---|
| *bandhic.save_npz*(file_name, mat) | Save a band_hic_matrix to a .npz file. |
| *bandhic.load_npz*(file_name) | Load a band_hic_matrix from a .npz file. |
| *bandhic.straw_chr*(hic_file, chrom, ...[, ...]) | Read Hi-C data from a .hic file and return a band_hic_matrix. |
| *bandhic.straw_all_chrs*(hic_file, resolution, ...) | Read Hi-C data from a .hic file for all chromosomes and return a dictionary of band_hic_matrix objects. |
| *bandhic.cooler_chr*(file_path, chrom, diag_num) | Read Hi-C data from a .cool or .mcool file and return a band_hic_matrix. |
| *bandhic.cooler_all_chrs*(file_path, diag_num) | Read Hi-C data from a .cool or .mcool file for all chromosomes and return a dictionary of band_hic_matrix objects. |
| *bandhic.cooler_chr_all_cells*(file_path, ...) | Read Hi-C data from a .scool file for a specific chromosome and return a dictionary of band_hic_matrix objects for all cells. |
| *bandhic.cooler_all_cells_all_chrs*(file_path, ...) | Read Hi-C data from a .scool file for all cells and return a dictionary of dictionaries of band_hic_matrix objects. |

### 3.3.1 bandhic.save_npz

bandhic.**save_npz**(*file_name*, *mat*)

Save a band_hic_matrix to a .npz file.

**Parameters**

- **file_name** (*str*) – Path to save the .npz file.
- **mat** (band_hic_matrix) – The band_hic_matrix object to save.

**Return type**
None

### Examples

```
>>> import bandhic as bh
>>> mat = bh.band_hic_matrix(np.eye(5), diag_num=3)
>>> save_npz('./test/sample.npz', mat)
```

## 3.3.2 bandhic.load_npz

bandhic.**load_npz**(*file_name*)

Load a band_hic_matrix from a .npz file.

**Parameters**
**file_name** (*str*) – Path to the .npz file.

**Returns**
A band_hic_matrix object loaded from the file.

**Return type**
*band_hic_matrix*

### Examples

```
>>> import bandhic as bh
>>> mat = bh.load_npz('./test/sample.npz')
>>> isinstance(mat, band_hic_matrix)
True
```

## 3.3.3 bandhic.straw_chr

bandhic.**straw_chr**(*hic_file*, *chrom*, *resolution*, *diag_num*, *data_type='observed'*, *normalization='NONE'*, *unit='BP'*)

Read Hi-C data from a .hic file and return a band_hic_matrix.

**Parameters**

- **hic_file** (*str*) – Path to the .hic file. This file should be in the Hi-C format compatible with hicstraw. Local or remote paths are supported.
- **chrom** (*str*) – Chromosome name (e.g., 'chr1', 'chrX'). Short names like '1', 'X' are also accepted.
- **resolution** (*int*) – Resolution of the Hi-C data. Such as 10000 for 10kb resolution.
- **diag_num** (*int*) – Number of diagonals to consider.
- **data_type** (*str, optional*) – Type of data to read from the Hi-C file. Default is 'observed'. Other options include 'expected', 'balanced', etc. See *hicstra`w* documentation for more details.
- **normalization** (*str, optional*) – Normalization method to apply. Default is 'NONE'. Other options include 'VC', 'VC_SQRT', 'KR', 'SCALE', etc. See *hicstraw* documentation for more details.
- **unit** (*str, optional*) – Unit of measurement for the Hi-C data. Default is 'BP' (base pairs). Other options include 'FRAG' (fragments), etc.

**Return type**
band_hic_matrix

**See also**

hicstraw documentation Python interface to read Hi-C data files using *.hic* format.

**Returns**
A band_hic_matrix object containing the Hi-C data.

**Return type**
*band_hic_matrix*

**Raises**
**ValueError** – If the file cannot be parsed or parameters are invalid.

**Parameters**

- **hic_file** (*str*)
- **chrom** (*str*)
- **resolution** (*int*)
- **diag_num** (*int*)
- **data_type** (*str*)
- **normalization** (*str*)
- **unit** (*str*)

### Examples

```
>>> import bandhic as bh
>>> mat = bh.straw_chr('/Users/wwb/Documents/workspace/BandHiC-Master/data/
↪GSE130275_mESC_WT_combined_1.3B_microc.hic', 'chr1', resolution=10000, diag_
↪num=200)
>>> isinstance(mat, band_hic_matrix)
True
```

## 3.3.4 bandhic.straw_all_chrs

bandhic.**straw_all_chrs**(*hic_file*, *resolution*, *diag_num*, *data_type='observed'*, *normalization='NONE'*, *unit='BP'*)

Read Hi-C data from a .hic file for all chromosomes and return a dictionary of band_hic_matrix objects.

**Parameters**

- **hic_file** (*str*) – Path to the .hic file. This file should be in the Hi-C format compatible with hicstraw. Local or remote paths are supported.
- **resolution** (*int*) – Resolution of the Hi-C data. Such as 10000 for 10kb resolution.
- **diag_num** (*int*) – Number of diagonals to consider.
- **data_type** (*str, optional*) – Type of data to read from the Hi-C file. Default is 'observed'. Other options include 'expected', 'balanced', etc. See *hicstraw* documentation for more details.

- **normalization** (*str, optional*) – Normalization method to apply. Default is 'NONE'. Other options include 'VC', 'VC_SQRT', 'KR', 'SCALE', etc. See *hicstraw* documentation for more details.
- **unit** (*str, optional*) – Unit of measurement for the Hi-C data. Default is 'BP' (base pairs). Other options include 'FRAG' (fragments), etc.

**Returns**
A dictionary mapping chromosome names to band_hic_matrix objects containing the Hi-C data.

**Return type**
Dict[str, *band_hic_matrix*]

**Raises**
**ValueError** – If the file cannot be parsed or parameters are invalid.

### Examples

```
>>> import bandhic as bh
>>> mats = bh.straw_all_chrs('/Users/wwb/Documents/workspace/BandHiC-Master/data/
↪GSE130275_mESC_WT_combined_1.3B_microc.hic', resolution=10000, diag_num=200)
>>> isinstance(mats['chr1'], band_hic_matrix)
True
```

## 3.3.5 bandhic.cooler_chr

bandhic.**cooler_chr**(*file_path*, *chrom*, *diag_num*, *cell_id=None*, *resolution=None*, *balance=True*)

Read Hi-C data from a .cool or .mcool file and return a band_hic_matrix.

**Parameters**

- **file_path** (*str*) – Path to the .cool, .mcool or .scool file.
- **chrom** (*str*) – Chromosome name.
- **diag_num** (*int*) – Number of diagonals to consider.
- **cell_id** (*str, optional*) – Cell ID for .scool files.
- **resolution** (*int, optional*) – Resolution of the Hi-C data.
- **balance** (*bool, optional*) – If True, use balanced data. Default is False. This parameter is specific to cooler files.

**Returns**
A band_hic_matrix object containing the Hi-C data.

**Return type**
*band_hic_matrix*

**Raises**
**ValueError** – If the cooler file is invalid or parameters are incorrect.

### Examples

```
>>> import bandhic as bh
>>> mat = bh.cooler_chr('/Users/wwb/Documents/workspace/BandHiC-Master/data/yeast.
↪10kb.cool', 'chrI', resolution=10000, diag_num=10)
>>> isinstance(mat, band_hic_matrix)
True
```

**See also**

cooler documentation Official documentation for the Cooler format and API usage.

### 3.3.6 bandhic.cooler_all_chrs

bandhic.**cooler_all_chrs**(*file_path*, *diag_num*, *resolution=None*, *cell_id=None*, *balance=True*)

Read Hi-C data from a .cool or .mcool file for all chromosomes and return a dictionary of band_hic_matrix objects.

**Parameters**

- **file_path** (*str*) – Path to the .cool, .mcool or .scool file.
- **diag_num** (*int*) – Number of diagonals to consider.
- **resolution** (*int, optional*) – Resolution of the Hi-C data.
- **cell_id** (*str, optional*) – Cell ID for .scool files.
- **balance** (*bool, optional*) – If True, use balanced data. Default is False. This parameter is specific to cooler files.

**Returns**

A dictionary mapping chromosome names to band_hic_matrix objects containing the Hi-C data.

**Return type**

Dict[str, *band_hic_matrix*]

**Raises**

**ValueError** – If the cooler file is invalid or parameters are incorrect.

#### Examples

```
>>> import bandhic as bh
>>> mats = bh.cooler_all_chrs('/Users/wwb/Documents/workspace/BandHiC-Master/data/
↪yeast.10kb.cool', diag_num=10, resolution=10000)
>>> isinstance(mats['chrI'], band_hic_matrix)
True
```

### 3.3.7 bandhic.cooler_chr_all_cells

bandhic.**cooler_chr_all_cells**(*file_path*, *chrom*, *diag_num*, *balance=True*)

Read Hi-C data from a .scool file for a specific chromosome and return a dictionary of band_hic_matrix objects for all cells.

**Parameters**

- **file_path** (*str*) – Path to the .scool file.
- **chrom** (*str*) – Chromosome name.
- **diag_num** (*int*) – Number of diagonals to consider.
- **balance** (*bool, optional*) – If True, use balanced data. Default is False. This parameter is specific to cooler files.

**Returns**

A dictionary mapping cell IDs to band_hic_matrix objects for the specified chromosome.

**Return type**
Dict[str, *band_hic_matrix*]

**Raises**
**ValueError** – If the scool file is invalid or parameters are incorrect.

### Examples

```
>>> import bandhic as bh
>>> mats = bh.cooler_chr_all_cells('/Users/wwb/Documents/workspace/BandHiC-Master/
↪data/yeast.10kb.scool', 'chrI', diag_num=10, resolution=10000)
>>> isinstance(mats['cell1'], band_hic_matrix)
True
```

## 3.3.8 bandhic.cooler_all_cells_all_chrs

bandhic.**cooler_all_cells_all_chrs**(*file_path*, *diag_num*, *resolution=None*)

Read Hi-C data from a .scool file for all cells and return a dictionary of dictionaries of band_hic_matrix objects.

**Parameters**

- **file_path** (*str*) – Path to the .scool file.
- **diag_num** (*int*) – Number of diagonals to consider.
- **resolution** (*int, optional*) – Resolution of the Hi-C data.

**Returns**
A dictionary mapping cell IDs to dictionaries mapping chromosome names to band_hic_matrix objects.

**Return type**
Dict[str, Dict[str, *band_hic_matrix*]]

**Raises**
**ValueError** – If the scool file is invalid or parameters are incorrect.

### Examples

```
>>> import bandhic as bh
>>> mats = bh.cooler_all_cells('/Users/wwb/Documents/workspace/BandHiC-Master/data/
↪yeast.10kb.scool', diag_num=10, resolution=10000)
>>> isinstance(mats['cell1']['chrI'], band_hic_matrix)
True
```

# 3.4 Vectorized operations for BandHiC matrices

These functions perform element-wise operations on BandHiC matrices, allowing for efficient computation across the matrix elements. These functions are wrapped around numpy ufuncs to provide a consistent interface for element-wise operations.

| | |
|---|---|
| *bandhic.absolute*(x, *args, **kwargs) | Perform element-wise 'absolute' operation. |
| *bandhic.add*(x1, x2, *args, **kwargs) | Perform element-wise 'add' operation with two inputs. |
| *bandhic.arccos*(x, *args, **kwargs) | Perform element-wise 'arccos' operation. |

continues on next page

Table 9 – continued from previous page

| | |
|---|---|
| bandhic.arccosh(x, *args, **kwargs) | Perform element-wise 'arccosh' operation. |
| bandhic.arcsin(x, *args, **kwargs) | Perform element-wise 'arcsin' operation. |
| bandhic.arcsinh(x, *args, **kwargs) | Perform element-wise 'arcsinh' operation. |
| bandhic.arctan(x, *args, **kwargs) | Perform element-wise 'arctan' operation. |
| bandhic.arctan2(x1, x2, *args, **kwargs) | Perform element-wise 'arctan2' operation with two inputs. |
| bandhic.arctanh(x, *args, **kwargs) | Perform element-wise 'arctanh' operation. |
| bandhic.bitwise_and(x1, x2, *args, **kwargs) | Perform element-wise 'bitwise_and' operation with two inputs. |
| bandhic.bitwise_or(x1, x2, *args, **kwargs) | Perform element-wise 'bitwise_or' operation with two inputs. |
| bandhic.bitwise_xor(x1, x2, *args, **kwargs) | Perform element-wise 'bitwise_xor' operation with two inputs. |
| bandhic.cbrt(x, *args, **kwargs) | Perform element-wise 'cbrt' operation. |
| bandhic.conj(x, *args, **kwargs) | Perform element-wise 'conj' operation. |
| bandhic.conjugate(x, *args, **kwargs) | Perform element-wise 'conjugate' operation. |
| bandhic.cos(x, *args, **kwargs) | Perform element-wise 'cos' operation. |
| bandhic.cosh(x, *args, **kwargs) | Perform element-wise 'cosh' operation. |
| bandhic.deg2rad(x, *args, **kwargs) | Perform element-wise 'deg2rad' operation. |
| bandhic.degrees(x, *args, **kwargs) | Perform element-wise 'degrees' operation. |
| bandhic.divide(x1, x2, *args, **kwargs) | Perform element-wise 'divide' operation with two inputs. |
| bandhic.divmod(x1, x2, *args, **kwargs) | Perform element-wise 'divmod' operation with two inputs. |
| bandhic.equal(x1, x2, *args, **kwargs) | Perform element-wise 'equal' operation with two inputs. |
| bandhic.exp(x, *args, **kwargs) | Perform element-wise 'exp' operation. |
| bandhic.exp2(x, *args, **kwargs) | Perform element-wise 'exp2' operation. |
| bandhic.expm1(x, *args, **kwargs) | Perform element-wise 'expm1' operation. |
| bandhic.fabs(x, *args, **kwargs) | Perform element-wise 'fabs' operation. |
| bandhic.float_power(x1, x2, *args, **kwargs) | Perform element-wise 'float_power' operation with two inputs. |
| bandhic.floor_divide(x1, x2, *args, **kwargs) | Perform element-wise 'floor_divide' operation with two inputs. |
| bandhic.fmod(x1, x2, *args, **kwargs) | Perform element-wise 'fmod' operation with two inputs. |
| bandhic.gcd(x1, x2, *args, **kwargs) | Perform element-wise 'gcd' operation with two inputs. |
| bandhic.greater(x1, x2, *args, **kwargs) | Perform element-wise 'greater' operation with two inputs. |
| bandhic.greater_equal(x1, x2, *args, **kwargs) | Perform element-wise 'greater_equal' operation with two inputs. |
| bandhic.heaviside(x1, x2, *args, **kwargs) | Perform element-wise 'heaviside' operation with two inputs. |
| bandhic.hypot(x1, x2, *args, **kwargs) | Perform element-wise 'hypot' operation with two inputs. |
| bandhic.invert(x, *args, **kwargs) | Perform element-wise 'invert' operation. |
| bandhic.lcm(x1, x2, *args, **kwargs) | Perform element-wise 'lcm' operation with two inputs. |
| bandhic.left_shift(x1, x2, *args, **kwargs) | Perform element-wise 'left_shift' operation with two inputs. |
| bandhic.less(x1, x2, *args, **kwargs) | Perform element-wise 'less' operation with two inputs. |
| bandhic.less_equal(x1, x2, *args, **kwargs) | Perform element-wise 'less_equal' operation with two inputs. |
| bandhic.log(x, *args, **kwargs) | Perform element-wise 'log' operation. |
| bandhic.log1p(x, *args, **kwargs) | Perform element-wise 'log1p' operation. |
| bandhic.log2(x, *args, **kwargs) | Perform element-wise 'log2' operation. |
| bandhic.log10(x, *args, **kwargs) | Perform element-wise 'log10' operation. |

continues on next page

Table 9 – continued from previous page

| | |
|---|---|
| bandhic.logaddexp(x1, x2, *args, **kwargs) | Perform element-wise 'logaddexp' operation with two inputs. |
| bandhic.logaddexp2(x1, x2, *args, **kwargs) | Perform element-wise 'logaddexp2' operation with two inputs. |
| bandhic.logical_and(x1, x2, *args, **kwargs) | Perform element-wise 'logical_and' operation with two inputs. |
| bandhic.logical_or(x1, x2, *args, **kwargs) | Perform element-wise 'logical_or' operation with two inputs. |
| bandhic.logical_xor(x1, x2, *args, **kwargs) | Perform element-wise 'logical_xor' operation with two inputs. |
| bandhic.maximum(x1, x2, *args, **kwargs) | Perform element-wise 'maximum' operation with two inputs. |
| bandhic.minimum(x1, x2, *args, **kwargs) | Perform element-wise 'minimum' operation with two inputs. |
| bandhic.mod(x1, x2, *args, **kwargs) | Perform element-wise 'mod' operation with two inputs. |
| bandhic.multiply(x1, x2, *args, **kwargs) | Perform element-wise 'multiply' operation with two inputs. |
| bandhic.negative(x, *args, **kwargs) | Perform element-wise 'negative' operation. |
| bandhic.not_equal(x1, x2, *args, **kwargs) | Perform element-wise 'not_equal' operation with two inputs. |
| bandhic.positive(x, *args, **kwargs) | Perform element-wise 'positive' operation. |
| bandhic.power(x1, x2, *args, **kwargs) | Perform element-wise 'power' operation with two inputs. |
| bandhic.rad2deg(x, *args, **kwargs) | Perform element-wise 'rad2deg' operation. |
| bandhic.radians(x, *args, **kwargs) | Perform element-wise 'radians' operation. |
| bandhic.reciprocal(x, *args, **kwargs) | Perform element-wise 'reciprocal' operation. |
| bandhic.remainder(x1, x2, *args, **kwargs) | Perform element-wise 'remainder' operation with two inputs. |
| bandhic.right_shift(x1, x2, *args, **kwargs) | Perform element-wise 'right_shift' operation with two inputs. |
| bandhic.rint(x, *args, **kwargs) | Perform element-wise 'rint' operation. |
| bandhic.sign(x, *args, **kwargs) | Perform element-wise 'sign' operation. |
| bandhic.sin(x, *args, **kwargs) | Perform element-wise 'sin' operation. |
| bandhic.sinh(x, *args, **kwargs) | Perform element-wise 'sinh' operation. |
| bandhic.sqrt(x, *args, **kwargs) | Perform element-wise 'sqrt' operation. |
| bandhic.square(x, *args, **kwargs) | Perform element-wise 'square' operation. |
| bandhic.subtract(x1, x2, *args, **kwargs) | Perform element-wise 'subtract' operation with two inputs. |
| bandhic.tan(x, *args, **kwargs) | Perform element-wise 'tan' operation. |
| bandhic.tanh(x, *args, **kwargs) | Perform element-wise 'tanh' operation. |
| bandhic.true_divide(x1, x2, *args, **kwargs) | Perform element-wise 'true_divide' operation with two inputs. |

### 3.4.1 bandhic.absolute

bandhic.**absolute**(*x*, **args*, ***kwargs*)

Perform element-wise 'absolute' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.absolute.
- ****kwargs** (*dict*) – Keyword arguments for numpy.absolute.

**Returns**
Result of element-wise 'absolute' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.absolute

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.absolute(x1)
```

## 3.4.2 bandhic.add

bandhic.**add**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'add' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.add.
- ****kwargs** (*dict*) – Keyword arguments for numpy.add.

**Returns**
Result of element-wise 'add' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.add

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.add(x1, x2)
```

## 3.4.3 bandhic.arccos

bandhic.**arccos**(*x*, **args*, ***kwargs*)

Perform element-wise 'arccos' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.arccos.
- ****kwargs** (*dict*) – Keyword arguments for numpy.arccos.

**Returns**
Result of element-wise 'arccos' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.arccos

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.arccos(x1)
```

## 3.4.4 bandhic.arccosh

bandhic.**arccosh**(*x*, **args*, ***kwargs*)

Perform element-wise 'arccosh' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.arccosh.
- ****kwargs** (*dict*) – Keyword arguments for numpy.arccosh.

**Returns**
Result of element-wise 'arccosh' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.arccosh

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.arccosh(x1)
```

### 3.4.5 bandhic.arcsin

bandhic.**arcsin**(*x*, **args*, ***kwargs*)

Perform element-wise 'arcsin' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.arcsin.
- ****kwargs** (*dict*) – Keyword arguments for numpy.arcsin.

**Returns**

Result of element-wise 'arcsin' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.arcsin

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.arcsin(x1)
```

### 3.4.6 bandhic.arcsinh

bandhic.**arcsinh**(*x*, **args*, ***kwargs*)

Perform element-wise 'arcsinh' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.arcsinh.
- ****kwargs** (*dict*) – Keyword arguments for numpy.arcsinh.

**Returns**

Result of element-wise 'arcsinh' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.arcsinh

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.arcsinh(x1)
```

### 3.4.7 bandhic.arctan

bandhic.**arctan**(*x*, **args*, ***kwargs*)

Perform element-wise 'arctan' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.arctan.
- ****kwargs** (*dict*) – Keyword arguments for numpy.arctan.

**Returns**

Result of element-wise 'arctan' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.arctan

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.arctan(x1)
```

### 3.4.8 bandhic.arctan2

bandhic.**arctan2**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'arctan2' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.arctan2.
- ****kwargs** (*dict*) – Keyword arguments for numpy.arctan2.

**Returns**

Result of element-wise 'arctan2' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.arctan2

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.arctan2(x1, x2)
```

### 3.4.9 bandhic.arctanh

bandhic.**arctanh**(*x*, **args*, ***kwargs*)

Perform element-wise 'arctanh' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.arctanh.
- ****kwargs** (*dict*) – Keyword arguments for numpy.arctanh.

**Returns**

Result of element-wise 'arctanh' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.arctanh

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.arctanh(x1)
```

### 3.4.10 bandhic.bitwise_and

bandhic.**bitwise_and**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'bitwise_and' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.bitwise_and.
- ****kwargs** (*dict*) – Keyword arguments for numpy.bitwise_and.

**Returns**

Result of element-wise 'bitwise_and' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.bitwise_and

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.bitwise_and(x1, x2)
```

## 3.4.11 bandhic.bitwise_or

bandhic.**bitwise_or**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'bitwise_or' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.bitwise_or.
- ****kwargs** (*dict*) – Keyword arguments for numpy.bitwise_or.

**Returns**

Result of element-wise 'bitwise_or' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.bitwise_or

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.bitwise_or(x1, x2)
```

## 3.4.12 bandhic.bitwise_xor

bandhic.**bitwise_xor**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'bitwise_xor' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.bitwise_xor.

- ****kwargs** (*dict*) – Keyword arguments for numpy.bitwise_xor.

**Returns**
Result of element-wise 'bitwise_xor' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.bitwise_xor

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.bitwise_xor(x1, x2)
```

## 3.4.13 bandhic.cbrt

bandhic.**cbrt**(*x*, **args*, ***kwargs*)

Perform element-wise 'cbrt' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.cbrt.
- ****kwargs** (*dict*) – Keyword arguments for numpy.cbrt.

**Returns**
Result of element-wise 'cbrt' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.cbrt

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.cbrt(x1)
```

## 3.4.14 bandhic.conj

bandhic.**conj**(*x*, **args*, ***kwargs*)

Perform element-wise 'conj' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.conj.
- ****kwargs** (*dict*) – Keyword arguments for numpy.conj.

**Returns**
Result of element-wise 'conj' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.conj

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.conj(x1)
```

## 3.4.15 bandhic.conjugate

bandhic.**conjugate**(*x*, **args*, ***kwargs*)

Perform element-wise 'conjugate' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.conjugate.
- ****kwargs** (*dict*) – Keyword arguments for numpy.conjugate.

**Returns**
Result of element-wise 'conjugate' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.conjugate

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.conjugate(x1)
```

### 3.4.16 bandhic.cos

bandhic.**cos**(*x*, **args*, ***kwargs*)

Perform element-wise 'cos' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.cos.
- ****kwargs** (*dict*) – Keyword arguments for numpy.cos.

**Returns**

Result of element-wise 'cos' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.cos

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.cos(x1)
```

### 3.4.17 bandhic.cosh

bandhic.**cosh**(*x*, **args*, ***kwargs*)

Perform element-wise 'cosh' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.cosh.
- ****kwargs** (*dict*) – Keyword arguments for numpy.cosh.

**Returns**

Result of element-wise 'cosh' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.cosh

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.cosh(x1)
```

### 3.4.18 bandhic.deg2rad

bandhic.**deg2rad**(*x*, **args*, ***kwargs*)

Perform element-wise 'deg2rad' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.deg2rad.
- ****kwargs** (*dict*) – Keyword arguments for numpy.deg2rad.

**Returns**

Result of element-wise 'deg2rad' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.deg2rad

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.deg2rad(x1)
```

### 3.4.19 bandhic.degrees

bandhic.**degrees**(*x*, **args*, ***kwargs*)

Perform element-wise 'degrees' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.degrees.
- ****kwargs** (*dict*) – Keyword arguments for numpy.degrees.

**Returns**

Result of element-wise 'degrees' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.degrees

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.degrees(x1)
```

### 3.4.20 bandhic.divide

bandhic.**divide**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'divide' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.divide.
- ****kwargs** (*dict*) – Keyword arguments for numpy.divide.

**Returns**

Result of element-wise 'divide' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.divide

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.divide(x1, x2)
```

### 3.4.21 bandhic.divmod

bandhic.**divmod**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'divmod' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.divmod.
- ****kwargs** (*dict*) – Keyword arguments for numpy.divmod.

**Returns**

Result of element-wise 'divmod' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.divmod

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.divmod(x1, x2)
```

## 3.4.22 bandhic.equal

bandhic.**equal**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'equal' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.equal.
- ****kwargs** (*dict*) – Keyword arguments for numpy.equal.

**Returns**

Result of element-wise 'equal' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.equal

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.equal(x1, x2)
```

## 3.4.23 bandhic.exp

bandhic.**exp**(*x*, **args*, ***kwargs*)

Perform element-wise 'exp' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.exp.
- ****kwargs** (*dict*) – Keyword arguments for numpy.exp.

**Returns**

Result of element-wise 'exp' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.exp

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.exp(x1)
```

## 3.4.24 bandhic.exp2

bandhic.**exp2**(*x*, **args*, ***kwargs*)

Perform element-wise 'exp2' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.exp2.
- ****kwargs** (*dict*) – Keyword arguments for numpy.exp2.

**Returns**

Result of element-wise 'exp2' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.exp2

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.exp2(x1)
```

## 3.4.25 bandhic.expm1

bandhic.**expm1**(*x*, **args*, ***kwargs*)

Perform element-wise 'expm1' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.expm1.
- ****kwargs** (*dict*) – Keyword arguments for numpy.expm1.

**Returns**

Result of element-wise 'expm1' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.expm1

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.expm1(x1)
```

### 3.4.26 bandhic.fabs

bandhic.**fabs**(*x*, **args*, ***kwargs*)

Perform element-wise 'fabs' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.fabs.
- ****kwargs** (*dict*) – Keyword arguments for numpy.fabs.

**Returns**

Result of element-wise 'fabs' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.fabs

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.fabs(x1)
```

### 3.4.27 bandhic.float_power

bandhic.**float_power**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'float_power' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.float_power.
- ****kwargs** (*dict*) – Keyword arguments for numpy.float_power.

**Returns**

Result of element-wise 'float_power' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.float_power

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.float_power(x1, x2)
```

## 3.4.28 bandhic.floor_divide

bandhic.**floor_divide**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'floor_divide' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.floor_divide.
- ****kwargs** (*dict*) – Keyword arguments for numpy.floor_divide.

**Returns**
Result of element-wise 'floor_divide' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.floor_divide

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.floor_divide(x1, x2)
```

## 3.4.29 bandhic.fmod

bandhic.**fmod**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'fmod' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.

- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.fmod.
- ****kwargs** (*dict*) – Keyword arguments for numpy.fmod.

**Returns**
Result of element-wise 'fmod' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.fmod

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.fmod(x1, x2)
```

## 3.4.30 bandhic.gcd

bandhic.**gcd**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'gcd' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.gcd.
- ****kwargs** (*dict*) – Keyword arguments for numpy.gcd.

**Returns**
Result of element-wise 'gcd' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.gcd

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.gcd(x1, x2)
```

### 3.4.31 bandhic.greater

bandhic.**greater**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'greater' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.greater.
- ****kwargs** (*dict*) – Keyword arguments for numpy.greater.

**Returns**

Result of element-wise 'greater' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.greater

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.greater(x1, x2)
```

### 3.4.32 bandhic.greater_equal

bandhic.**greater_equal**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'greater_equal' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.greater_equal.
- ****kwargs** (*dict*) – Keyword arguments for numpy.greater_equal.

**Returns**

Result of element-wise 'greater_equal' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.greater_equal

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.greater_equal(x1, x2)
```

## 3.4.33 bandhic.heaviside

bandhic.**heaviside**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'heaviside' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.heaviside.
- ****kwargs** (*dict*) – Keyword arguments for numpy.heaviside.

**Returns**

Result of element-wise 'heaviside' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.heaviside

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.heaviside(x1, x2)
```

## 3.4.34 bandhic.hypot

bandhic.**hypot**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'hypot' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.hypot.
- ****kwargs** (*dict*) – Keyword arguments for numpy.hypot.

**Returns**

Result of element-wise 'hypot' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.hypot

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.hypot(x1, x2)
```

## 3.4.35 bandhic.invert

bandhic.**invert**(*x*, **args*, ***kwargs*)

Perform element-wise 'invert' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.invert.
- ****kwargs** (*dict*) – Keyword arguments for numpy.invert.

**Returns**

Result of element-wise 'invert' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.invert

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.invert(x1)
```

## 3.4.36 bandhic.lcm

bandhic.**lcm**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'lcm' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.lcm.
- ****kwargs** (*dict*) – Keyword arguments for numpy.lcm.

**Returns**
Result of element-wise 'lcm' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.lcm

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.lcm(x1, x2)
```

## 3.4.37 bandhic.left_shift

bandhic.**left_shift**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'left_shift' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.left_shift.
- ****kwargs** (*dict*) – Keyword arguments for numpy.left_shift.

**Returns**
Result of element-wise 'left_shift' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.left_shift

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.left_shift(x1, x2)
```

## 3.4.38 bandhic.less

bandhic.**less**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'less' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.less.
- ****kwargs** (*dict*) – Keyword arguments for numpy.less.

**Returns**
Result of element-wise 'less' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.less

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.less(x1, x2)
```

## 3.4.39 bandhic.less_equal

bandhic.**less_equal**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'less_equal' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.less_equal.
- ****kwargs** (*dict*) – Keyword arguments for numpy.less_equal.

**Returns**
Result of element-wise 'less_equal' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.less_equal

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.less_equal(x1, x2)
```

### 3.4.40 bandhic.log

bandhic.**log**(*x*, **args*, ***kwargs*)

Perform element-wise 'log' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.log.
- ****kwargs** (*dict*) – Keyword arguments for numpy.log.

**Returns**

Result of element-wise 'log' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.log

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.log(x1)
```

### 3.4.41 bandhic.log1p

bandhic.**log1p**(*x*, **args*, ***kwargs*)

Perform element-wise 'log1p' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.log1p.
- ****kwargs** (*dict*) – Keyword arguments for numpy.log1p.

**Returns**

Result of element-wise 'log1p' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.log1p

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.log1p(x1)
```

### 3.4.42 bandhic.log2

bandhic.**log2**(*x*, **args*, ***kwargs*)

Perform element-wise 'log2' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.log2.
- ****kwargs** (*dict*) – Keyword arguments for numpy.log2.

**Returns**

Result of element-wise 'log2' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.log2

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.log2(x1)
```

### 3.4.43 bandhic.log10

bandhic.**log10**(*x*, **args*, ***kwargs*)

Perform element-wise 'log10' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.log10.
- ****kwargs** (*dict*) – Keyword arguments for numpy.log10.

**Returns**

Result of element-wise 'log10' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.log10

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.log10(x1)
```

### 3.4.44 bandhic.logaddexp

bandhic.**logaddexp**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'logaddexp' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.logaddexp.
- ****kwargs** (*dict*) – Keyword arguments for numpy.logaddexp.

**Returns**

Result of element-wise 'logaddexp' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.logaddexp

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.logaddexp(x1, x2)
```

### 3.4.45 bandhic.logaddexp2

bandhic.**logaddexp2**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'logaddexp2' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.logaddexp2.
- ****kwargs** (*dict*) – Keyword arguments for numpy.logaddexp2.

**Returns**

Result of element-wise 'logaddexp2' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.logaddexp2

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.logaddexp2(x1, x2)
```

## 3.4.46 bandhic.logical_and

bandhic.**logical_and**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'logical_and' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.logical_and.
- ****kwargs** (*dict*) – Keyword arguments for numpy.logical_and.

**Returns**

Result of element-wise 'logical_and' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.logical_and

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.logical_and(x1, x2)
```

## 3.4.47 bandhic.logical_or

bandhic.**logical_or**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'logical_or' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.logical_or.
- ****kwargs** (*dict*) – Keyword arguments for numpy.logical_or.

**Returns**

Result of element-wise 'logical_or' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.logical_or

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.logical_or(x1, x2)
```

## 3.4.48 bandhic.logical_xor

bandhic.**logical_xor**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'logical_xor' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.logical_xor.
- ****kwargs** (*dict*) – Keyword arguments for numpy.logical_xor.

**Returns**

Result of element-wise 'logical_xor' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.logical_xor

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.logical_xor(x1, x2)
```

## 3.4.49 bandhic.maximum

bandhic.**maximum**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'maximum' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.maximum.

- ****kwargs** (*dict*) – Keyword arguments for numpy.maximum.

**Returns**
Result of element-wise 'maximum' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.maximum

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.maximum(x1, x2)
```

## 3.4.50 bandhic.minimum

bandhic.**minimum**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'minimum' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.minimum.
- ****kwargs** (*dict*) – Keyword arguments for numpy.minimum.

**Returns**
Result of element-wise 'minimum' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.minimum

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.minimum(x1, x2)
```

### 3.4.51 bandhic.mod

bandhic.**mod**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'mod' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.mod.
- ****kwargs** (*dict*) – Keyword arguments for numpy.mod.

**Returns**

Result of element-wise 'mod' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.mod

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.mod(x1, x2)
```

### 3.4.52 bandhic.multiply

bandhic.**multiply**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'multiply' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.multiply.
- ****kwargs** (*dict*) – Keyword arguments for numpy.multiply.

**Returns**

Result of element-wise 'multiply' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.multiply

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.multiply(x1, x2)
```

## 3.4.53 bandhic.negative

bandhic.**negative**(*x*, **args*, ***kwargs*)

Perform element-wise 'negative' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.negative.
- ****kwargs** (*dict*) – Keyword arguments for numpy.negative.

**Returns**

Result of element-wise 'negative' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.negative

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.negative(x1)
```

## 3.4.54 bandhic.not_equal

bandhic.**not_equal**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'not_equal' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.not_equal.
- ****kwargs** (*dict*) – Keyword arguments for numpy.not_equal.

**Returns**

Result of element-wise 'not_equal' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.not_equal

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.not_equal(x1, x2)
```

### 3.4.55 bandhic.positive

bandhic.**positive**(*x*, **args*, ***kwargs*)

Perform element-wise 'positive' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.positive.
- ****kwargs** (*dict*) – Keyword arguments for numpy.positive.

**Returns**

Result of element-wise 'positive' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.positive

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.positive(x1)
```

### 3.4.56 bandhic.power

bandhic.**power**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'power' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.power.
- ****kwargs** (*dict*) – Keyword arguments for numpy.power.

**Returns**
Result of element-wise 'power' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.power

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.power(x1, x2)
```

## 3.4.57 bandhic.rad2deg

bandhic.**rad2deg**(*x*, **args*, ***kwargs*)

Perform element-wise 'rad2deg' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.rad2deg.
- ****kwargs** (*dict*) – Keyword arguments for numpy.rad2deg.

**Returns**
Result of element-wise 'rad2deg' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.rad2deg

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.rad2deg(x1)
```

## 3.4.58 bandhic.radians

bandhic.**radians**(*x*, **args*, ***kwargs*)

Perform element-wise 'radians' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.radians.

- ****kwargs** (*dict*) – Keyword arguments for numpy.radians.

**Returns**
Result of element-wise 'radians' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.radians

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.radians(x1)
```

## 3.4.59 bandhic.reciprocal

bandhic.**reciprocal**(*x*, **args*, ***kwargs*)

Perform element-wise 'reciprocal' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.reciprocal.
- ****kwargs** (*dict*) – Keyword arguments for numpy.reciprocal.

**Returns**
Result of element-wise 'reciprocal' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.reciprocal

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.reciprocal(x1)
```

## 3.4.60 bandhic.remainder

bandhic.**remainder**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'remainder' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.

- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.remainder.
- ****kwargs** (*dict*) – Keyword arguments for numpy.remainder.

**Returns**
Result of element-wise 'remainder' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.remainder

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.remainder(x1, x2)
```

## 3.4.61 bandhic.right_shift

bandhic.**right_shift**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'right_shift' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.right_shift.
- ****kwargs** (*dict*) – Keyword arguments for numpy.right_shift.

**Returns**
Result of element-wise 'right_shift' operation.

**Return type**
*band_hic_matrix*

**See also**

numpy.right_shift

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.right_shift(x1, x2)
```

### 3.4.62 bandhic.rint

bandhic.**rint**(*x*, **args*, ***kwargs*)

Perform element-wise 'rint' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.rint.
- ****kwargs** (*dict*) – Keyword arguments for numpy.rint.

**Returns**

Result of element-wise 'rint' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.rint

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.rint(x1)
```

### 3.4.63 bandhic.sign

bandhic.**sign**(*x*, **args*, ***kwargs*)

Perform element-wise 'sign' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.sign.
- ****kwargs** (*dict*) – Keyword arguments for numpy.sign.

**Returns**

Result of element-wise 'sign' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.sign

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.sign(x1)
```

### 3.4.64 bandhic.sin

bandhic.**sin**(*x*, **args*, ***kwargs*)

Perform element-wise 'sin' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.sin.
- ****kwargs** (*dict*) – Keyword arguments for numpy.sin.

**Returns**

Result of element-wise 'sin' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.sin

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.sin(x1)
```

### 3.4.65 bandhic.sinh

bandhic.**sinh**(*x*, **args*, ***kwargs*)

Perform element-wise 'sinh' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.sinh.
- ****kwargs** (*dict*) – Keyword arguments for numpy.sinh.

**Returns**

Result of element-wise 'sinh' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.sinh

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.sinh(x1)
```

### 3.4.66 bandhic.sqrt

bandhic.**sqrt**(*x*, **args*, ***kwargs*)

Perform element-wise 'sqrt' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.sqrt.
- ****kwargs** (*dict*) – Keyword arguments for numpy.sqrt.

**Returns**

Result of element-wise 'sqrt' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.sqrt

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.sqrt(x1)
```

### 3.4.67 bandhic.square

bandhic.**square**(*x*, **args*, ***kwargs*)

Perform element-wise 'square' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.square.
- ****kwargs** (*dict*) – Keyword arguments for numpy.square.

**Returns**

Result of element-wise 'square' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.square

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.square(x1)
```

### 3.4.68 bandhic.subtract

bandhic.**subtract**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'subtract' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.subtract.
- ****kwargs** (*dict*) – Keyword arguments for numpy.subtract.

**Returns**

Result of element-wise 'subtract' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.subtract

#### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.subtract(x1, x2)
```

### 3.4.69 bandhic.tan

bandhic.**tan**(*x*, **args*, ***kwargs*)

Perform element-wise 'tan' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.tan.
- ****kwargs** (*dict*) – Keyword arguments for numpy.tan.

**Returns**

Result of element-wise 'tan' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.tan

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.tan(x1)
```

## 3.4.70 bandhic.tanh

bandhic.**tanh**(*x*, **args*, ***kwargs*)

Perform element-wise 'tanh' operation.

**Parameters**

- **x** (band_hic_matrix) – Input matrix.
- ***args** (*tuple*) – Additional positional arguments for numpy.tanh.
- ****kwargs** (*dict*) – Keyword arguments for numpy.tanh.

**Returns**

Result of element-wise 'tanh' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.tanh

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> result = bh.tanh(x1)
```

## 3.4.71 bandhic.true_divide

bandhic.**true_divide**(*x1*, *x2*, **args*, ***kwargs*)

Perform element-wise 'true_divide' operation with two inputs.

**Parameters**

- **x1** (band_hic_matrix, *ndarray or scalar*) – First input matrix.
- **x2** (band_hic_matrix, *ndarray or scalar*) – Second input for the operation.
- ***args** (*tuple*) – Additional positional arguments for numpy.true_divide.
- ****kwargs** (*dict*) – Keyword arguments for numpy.true_divide.

**Returns**

Result of element-wise 'true_divide' operation.

**Return type**

*band_hic_matrix*

**See also**

numpy.true_divide

### Examples

```
>>> import bandhic as bh
>>> x1 = band_hic_matrix(np.eye(3), diag_num=2, dtype=int)
>>> x2 = x1.copy()
>>> result = bh.true_divide(x1, x2)
```

## 3.5 Data Reduction and Normalization

These functions perform data reduction and normalization on BandHiC matrices, allowing for efficient analysis of Hi-C data. These functions are wrapped around band_hic_matrix methods to provide a consistent interface for data reduction and normalization.

| | |
|---|---|
| *bandhic.min*(*args, **kwargs) | Wrapped from *band_hic_matrix.min* |
| *bandhic.max*(*args, **kwargs) | Wrapped from *band_hic_matrix.max* |
| *bandhic.sum*(*args, **kwargs) | Wrapped from *band_hic_matrix.sum* |
| *bandhic.mean*(*args, **kwargs) | Wrapped from *band_hic_matrix.mean* |
| *bandhic.std*(*args, **kwargs) | Wrapped from *band_hic_matrix.std* |
| *bandhic.var*(*args, **kwargs) | Wrapped from *band_hic_matrix.var* |
| *bandhic.prod*(*args, **kwargs) | Wrapped from *band_hic_matrix.prod* |
| *bandhic.ptp*(*args, **kwargs) | Wrapped from *band_hic_matrix.ptp* |
| *bandhic.all*(*args, **kwargs) | Wrapped from *band_hic_matrix.all* |
| *bandhic.any*(*args, **kwargs) | Wrapped from *band_hic_matrix.any* |
| *bandhic.clip*(*args, **kwargs) | Wrapped from *band_hic_matrix.clip* |
| *bandhic.normalize*(*args, **kwargs) | Wrapped from *band_hic_matrix.normalize* |

### 3.5.1 bandhic.min

bandhic.**min**(**args*, ***kwargs*)

Wrapped from *band_hic_matrix.min*

Refer to the documentation: *bandhic.band_hic_matrix.min()*

### 3.5.2 bandhic.max

bandhic.**max**(**args*, ***kwargs*)

Wrapped from *band_hic_matrix.max*

Refer to the documentation: *bandhic.band_hic_matrix.max()*

### 3.5.3 bandhic.sum

bandhic.**sum**(**args*, ***kwargs*)

Wrapped from *band_hic_matrix.sum*

Refer to the documentation: *bandhic.band_hic_matrix.sum()*

### 3.5.4 bandhic.mean

bandhic.**mean**(**args*, ***kwargs*)

Wrapped from *band_hic_matrix.mean*

Refer to the documentation: *bandhic.band_hic_matrix.mean()*

### 3.5.5 bandhic.std

bandhic.**std**(**args*, ***kwargs*)

Wrapped from *band_hic_matrix.std*

Refer to the documentation: *bandhic.band_hic_matrix.std()*

### 3.5.6 bandhic.var

bandhic.**var**(**args*, ***kwargs*)

Wrapped from *band_hic_matrix.var*

Refer to the documentation: *bandhic.band_hic_matrix.var()*

### 3.5.7 bandhic.prod

bandhic.**prod**(**args*, ***kwargs*)

Wrapped from *band_hic_matrix.prod*

Refer to the documentation: *bandhic.band_hic_matrix.prod()*

### 3.5.8 bandhic.ptp

bandhic.**ptp**(**args*, ***kwargs*)

Wrapped from *band_hic_matrix.ptp*

Refer to the documentation: *bandhic.band_hic_matrix.ptp()*

### 3.5.9 bandhic.all

bandhic.**all**(**args*, ***kwargs*)

Wrapped from *band_hic_matrix.all*

Refer to the documentation: *bandhic.band_hic_matrix.all()*

### 3.5.10 bandhic.any

bandhic.**any**(**args*, ***kwargs*)

Wrapped from *band_hic_matrix.any*

Refer to the documentation: *bandhic.band_hic_matrix.any()*

### 3.5.11 bandhic.clip

bandhic.**clip**(**args*, ***kwargs*)

Wrapped from *band_hic_matrix.clip*

Refer to the documentation: *bandhic.band_hic_matrix.clip()*

### 3.5.12 bandhic.normalize

bandhic.**normalize**(**args*, ***kwargs*)

Wrapped from *band_hic_matrix.normalize*

Refer to the documentation: *bandhic.band_hic_matrix.normalize()*

## 3.6 Other Functions

These functions provide additional utilities for BandHiC matrices, including matrix comparison, bias computation, and TAD calling.

| | |
|---|---|
| *bandhic.matrix_equal*(a, b) | Check if two band_hic_matrix objects are equal. |
| *bandhic.assert_band_matrix_equal*(a, b) | Assert that two band_hic_matrix objects are equal. |
| *bandhic.compute_bin_bias*(hic_coo[, verbose]) | Compute bias values for Hi-C contact matrices using the Knight-Ruiz normalization algorithm. |
| *bandhic.call_tad*(hic_matrix, resolution, ...) | Detect TADs from a Hi-C matrix using the TopDom algorithm. |
| *bandhic.topdom*(hic_matrix, bins, window_size) | Detect TADs using TopDom algorithm. |

### 3.6.1 bandhic.matrix_equal

bandhic.**matrix_equal**(*a*, *b*)

Check if two band_hic_matrix objects are equal.

**Parameters**

- **a** (band_hic_matrix) – First band_hic_matrix object.
- **b** (band_hic_matrix) – Second band_hic_matrix object.

**Returns**

True if the matrices are equal, False otherwise.

**Return type**

bool

### 3.6.2 bandhic.assert_band_matrix_equal

bandhic.**assert_band_matrix_equal**(*a*, *b*)

Assert that two band_hic_matrix objects are equal.

**Parameters**

- **a** (band_hic_matrix) – First band_hic_matrix object.
- **b** (band_hic_matrix) – Second band_hic_matrix object.

**Raises**

**AssertionError** – If the two matrices are not equal.

**Return type**

bool

### 3.6.3 bandhic.compute_bin_bias

bandhic.**compute_bin_bias**(*hic_coo*, *verbose=False*)

Compute bias values for Hi-C contact matrices using the Knight-Ruiz normalization algorithm.

**Parameters**

- **hic_coo** (*scipy.sparse.coo_array*) – A sparse COO matrix representing Hi-C contact data.
- **verbose** (*bool, optional*) – If True, print detailed information during processing. Default is False.

**Returns**

- **bias** (*numpy.ndarray*) – A 1D array containing the bias values for each bin in the Hi-C matrix.
- **is_valid** (*bool*) – A boolean indicating whether the bias vector is valid (mean and median within typical range).

#### Notes

This function removes a specified percentage of the most sparse bins from the Hi-C matrix before computing the bias values. The Knight-Ruiz normalization algorithm is applied to the modified matrix to compute the bias. The function iteratively removes bins with low interaction counts until a valid bias vector is obtained. The bias vector is expected to have a mean and median close to 1, indicating balanced interaction frequencies across bins.

### 3.6.4 bandhic.call_tad

bandhic.**call_tad**(*hic_matrix*, *resolution*, *chrom_short*, *window_size_bp=200000*, *min_TAD_size=None*, *stat_filter=True*, *verbose=False*)

Detect TADs from a Hi-C matrix using the TopDom algorithm. Different from the *TopDom* function, this function accepts a Hi-C matrix in either *band_hic_matrix* format or as a dense numpy array. This function is designed to be used with the BandHiC package and provides a convenient interface for TAD detection.

**Parameters**

- **hic_matrix** (band_hic_matrix *or np.ndarray*) – The Hi-C matrix, either as a band_hic_matrix object or a dense numpy array.
- **resolution** (*int*) – The resolution of the Hi-C matrix.
- **chrom_short** (*str*) – The chromosome name without 'chr' prefix.
- **window_size_bp** (*int, optional*) – The size of the window to consider for detecting TADs, in base pairs. Default is 200000.
- **min_TAD_size** (*int, optional*) – The minimum size of a TAD to be considered valid, in base pairs. Default is None.
- **stat_filter** (*bool, optional*) – Whether to apply statistical filtering to remove false positives. Default is True.
- **verbose** (*bool, optional*) – Whether to print detailed information during processing. Default is False.

**Returns**

- **domains** (*pd.DataFrame*) – DataFrame containing detected TADs with columns 'chr', 'from.id', 'from.coord', 'to.id', 'to.coord', 'tag'.
- **bins** (*pd.DataFrame*) – DataFrame containing bin information columns 'id', 'chr', 'from.coord', 'to.coord', 'local.ext', 'mean.cf', 'pvalue'.

### 3.6.5 bandhic.topdom

bandhic.**topdom**(*hic_matrix*, *bins*, *window_size*, *stat_filter=True*, *verbose=False*)

Detect TADs using TopDom algorithm.

**Parameters**

- **hic_matrix** (band_hic_matrix *or np.ndarray*) – The Hi-C matrix, either as a band_hic_matrix object or a dense numpy array.
- **bins** (*pd.DataFrame*) – DataFrame containing bin information with columns 'chr', 'from.coord', 'to.coord'.
- **window_size** (*int*) – Size of the window to consider for detecting TADs.
- **stat_filter** (*bool, optional*) – Whether to apply statistical filtering to remove false positives. Default is True.
- **verbose** (*bool, optional*) – Whether to print detailed information during processing. Default is False.

**Returns**

- **domains** (*pd.DataFrame*) – DataFrame containing detected TADs with columns 'chr', 'from.id', 'from.coord', 'to.id', 'to.coord', 'tag'.
- **bins** (*pd.DataFrame*) – Updated DataFrame containing bin information with additional columns 'local.ext', 'mean.cf', 'pvalue'.

CHAPTER

FOUR

# INDICES AND TABLES

- genindex
- modindex
- search

# INDEX

## Symbols



## A



## B



## C

## D



## E



## F



## G



## H



## I



## L